AMANAI LAB

LangChain Complete Course

Video 3 Notes

Data and Retrievers: Loaders, Splitters, Embeddings, Vector Stores, RAG and a Support Knowledge-Base App

Beginner to advanced · Theory, code and examples · Exercises and interview questions

Written for Python 3.10 or newer, langchain 1.4, langchain-core 1.6, langchain-groq 1.1 and the model **openai/gpt-oss-20b** on Groq (checked October 2026). Library versions and model lists change, so use the links at the end if something on your screen is different.

Contents

Start here	2
Part 1: Why RAG?	4
Part 2: Documents and loaders	10
Part 3: Text splitters	14
Part 4: Embeddings	19
Part 5: Vector stores	23
Part 6: Retrievers	30
Part 7: Build the RAG chain	36
Part 8: Project: the Nova Store policy assistant	42
Part 9: Making RAG better	48
Part 10: Common errors and how to debug them	50
Part 11: Practice, interview questions, cheat sheet and recap	53

Beginner start here **Intermediate** needed for real projects **Advanced** extra depth

Start here

What you will learn

This is Video 3 of the LangChain series. So far the model only knows what it learned in training and what you type into the prompt. In this video you connect it to **your own documents**. By the end, you can:

1. Explain what **RAG** (Retrieval-Augmented Generation) is and why it is used.
2. Describe the two phases of RAG: **indexing** (once) and **querying** (every question).
3. Load text, Markdown, PDF and CSV files with **document loaders**.
4. Cut documents into good pieces with **text splitters**.
5. Explain **embeddings** and compare pieces of text by meaning.
6. Store and search vectors with a **vector store** (in-memory, Chroma and FAISS).
7. Use **retrievers**: top-k, MMR, score threshold, metadata filters and keyword (BM25) search.
8. Build a complete **RAG chain** that answers from your documents and shows its sources.
9. Add chat history to a RAG chain.
10. Build a **Nova Store policy assistant** and upgrade the Video 2 ticket assistant with real policy knowledge.
11. Check retrieval quality and debug the most common RAG errors.

Who this is for

You finished Videos 1 and 2 (or you know `ChatGroq`, prompt templates, output parsers and the `|` pipe). You need basic Python: functions, lists, dictionaries and `try/except`. You do **not** need to know anything about vectors or databases. Both are explained from zero.

How to read these notes

- Every section has a **level tag**: Beginner, Intermediate or Advanced. Read the Beginner sections first.
- **Example output** boxes show what you may see. Your own text will be different, because AI answers change from run to run. Output of scripts that never call a model or download one (for example `make_data.py` and the splitter demos) is real and will match yours. Scores and answers from real embedding models and from Groq will differ a little.
- The first line of each code file (for example `# make_data.py`) tells you the file name.
- Use the **same project folder and virtual environment** as Videos 1 and 2. This video needs a few new packages (section 1.6).

Tag	Meaning
Beginner	Start here. No earlier knowledge needed
Intermediate	Builds on the beginner sections. Needed for real projects
Advanced	Extra depth. Useful once the basics feel easy

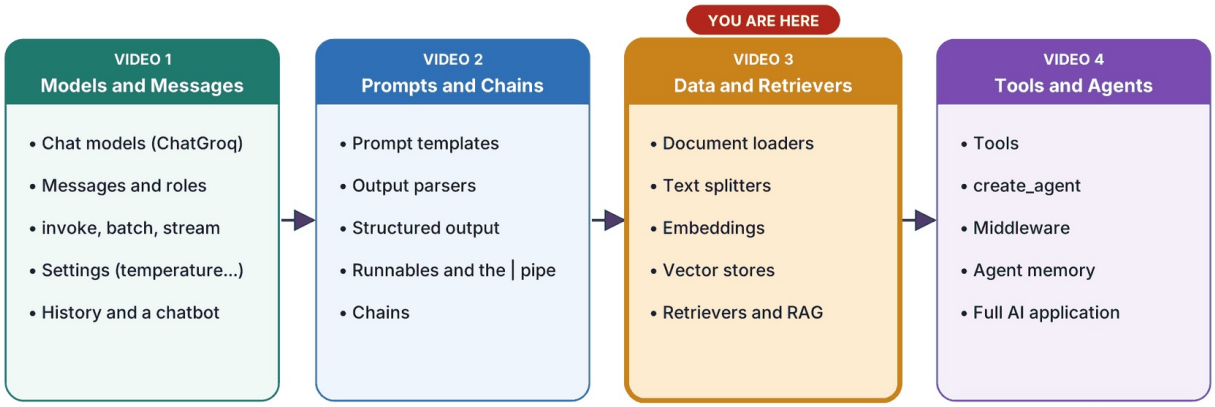
Files you will create

```
langchain-video-1/            (same folder as Videos 1 and 2)
├── .env                      # your GROQ_API_KEY (already there)
├── llm.py                    # helper from Video 1 (reused)
├── embed.py                  # Part 4: one place to create the embedding model
├── kb.py                     # Part 5: load + split helpers for our documents
├── store.py                  # Part 5: opens (and fills) our Chroma knowledge base
├── data/                     # created by make_data.py: our sample documents
├── no_context.py             # Part 1: the problem RAG solves
├── make_data.py              # Part 1: creates the sample documents
├── document_demo.py          # Part 2: the Document object
├── loaders_demo.py           # Part 2: text and directory loaders
├── pdf_csv_demo.py           # Part 2: PDF and CSV loaders
```

```
├── splitter_demo.py          # Part 3: chunk size and overlap
├── splitter_structure.py     # Part 3: Markdown and code splitters
├── fake_embed_demo.py        # Part 4: the embedding interface (no download)
├── embeddings_demo.py        # Part 4: real embeddings and similarity
├── vectorstore_memory.py     # Part 5: in-memory vector store
├── chroma_demo.py            # Part 5: Chroma, saved on disk
├── faiss_demo.py             # Part 5: FAISS
├── retriever_demo.py         # Part 6: top-k, MMR, threshold, filters
├── hybrid_demo.py            # Part 6: keyword + vector search
├── multiquery_demo.py        # Part 6: let the model rewrite the query
├── custom_retriever.py       # Part 6: write your own retriever
├── rag_common.py             # Part 7: prompt and formatting shared by RAG scripts
├── rag_basic.py              # Part 7: the first RAG chain
├── rag_sources.py            # Part 7: answer plus sources
├── rag_chat.py               # Part 7: RAG with chat history
├── build_index.py            # Part 8: build the knowledge base once
├── policy_bot.py             # Part 8: the policy assistant
├── ticket_rag.py             # Part 8: ticket assistant with policy knowledge
├── check_retrieval.py        # Part 8: measure retrieval quality
└── errors_demo3.py           # Part 10: common RAG errors
```

Part 1: Why RAG?

> **On the map:** you now move to the **Data** branch (Loaders, Documents, Splitters) and the **Retrievers** block. The model from Video 1 and the chains from Video 2 are the pieces that finish the job.



You are here: Video 3 covers Data and Retrievers.

1.1 Quick recap of Video 2 Beginner

Five ideas from Video 2 are used all the time in this video.

- A **prompt template** is a reusable prompt with named blanks. It needs a dictionary.
- A **chain** connects blocks with the pipe: `prompt | model | parser`.
- Every block is a **Runnable**, so it has `invoke`, `batch` and `stream`.
- `RunnablePassthrough` and `RunnableLambda` carry data forward and run your own Python.
- `with_structured_output` returns a validated object instead of text.

1.2 The problem: the model does not know your data Beginner

A chat model learned from public text up to a fixed date. It has never seen your refund policy, your wiki or your PDFs. Create `no_context.py` and ask it a question that only Nova Store (our imaginary online shop) can answer:

```
# no_context.py
from llm import get_model

model = get_model()

question = "How many days do I have to ask for a refund at Nova Store?"
answer = model.invoke(question)
print(answer.text)
```

Run it:

```
python no_context.py
```

Illustrative output:

```
Most online stores allow refunds within 30 days of purchase. Please check the
Nova Store website for the exact policy.
```

The model cannot know the real answer. It either guesses (a **hallucination**) or says it does not know. Both are useless for a support assistant. A wrong refund promise is worse than no answer.

1.3 Why not paste all documents into the prompt? Beginner

A natural idea: put every document into the prompt. It works for tiny data and fails for real data.

Problem	What happens
Context window	Every model has a maximum prompt size. A company wiki or a few PDFs will not fit
Cost and speed	You pay for every token, on every question. A bigger prompt is slower too
Noise	The model must find one useful sentence among thousands. Answers get worse
Freshness	When a document changes, you must rebuild the prompt by hand
Control	You cannot tell which part of the text the answer came from

1.4 The idea: Retrieval-Augmented Generation Beginner

RAG means: for each question, first **find** only the few pieces of text that matter, then **put them in the prompt**, then let the model answer from them.

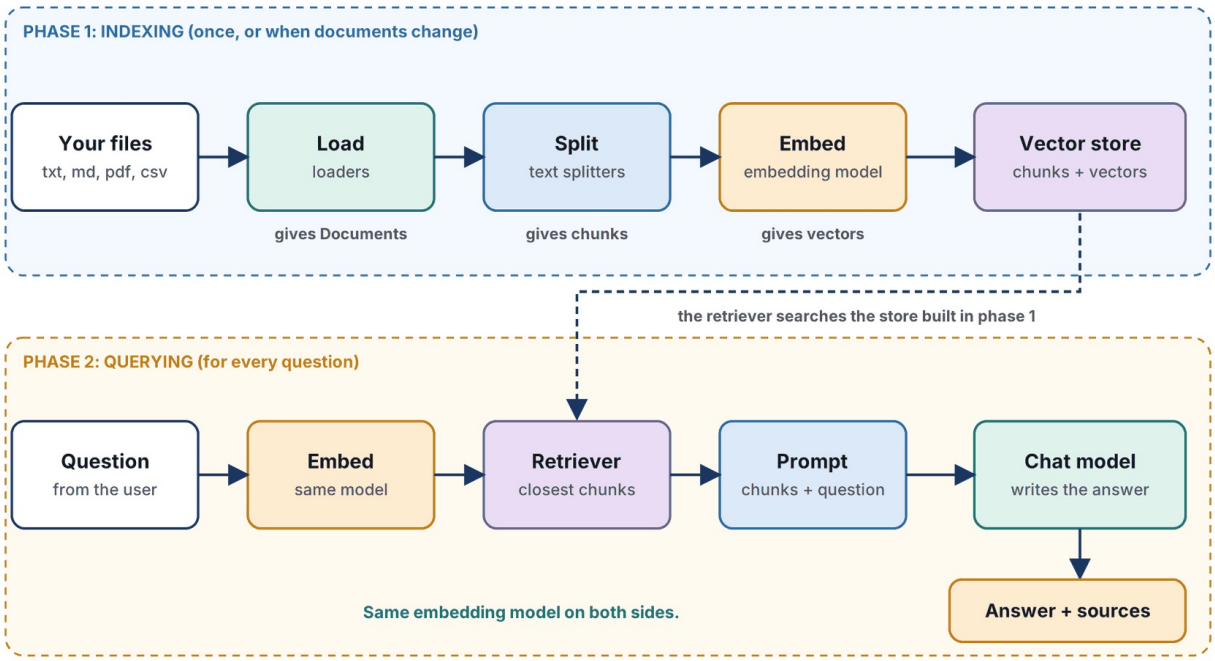
> **Analogy:** an open-book exam. You do not memorise the whole textbook. You keep it on the desk, find the right page quickly and write the answer from that page.

The three words in the name map to three jobs:

Word	Job	Done by
Retrieval	Find the relevant text	A retriever and a vector store
Augmented	Add that text to the prompt	A prompt template
Generation	Write the answer	The chat model

1.5 The RAG pipeline: two phases Beginner

RAG has two phases. Learn this picture well. The rest of the video explains one box at a time.



RAG in two phases. Indexing happens once. Querying happens for every question.

Phase 1: Indexing (once, or when documents change).

1. **Load** files into `Document` objects (document loaders).
2. **Split** long documents into small chunks (text splitters).

3. **Embed** each chunk into a list of numbers that captures its meaning (embedding model).
4. **Store** the chunks and their vectors (vector store).

Phase 2: Querying (every question).

1. **Embed** the question with the same embedding model.
2. **Search** the vector store for the chunks closest in meaning (retriever).
3. **Build the prompt** from the chunks and the question (prompt template).
4. **Generate** the answer with the chat model and show the sources.

Step	LangChain block	Part of these notes
Load	Document loaders	Part 2
Split	Text splitters	Part 3
Embed	Embedding models	Part 4
Store	Vector stores	Part 5
Retrieve	Retrievers	Part 6
Generate	Prompt, model, parser (a chain)	Part 7

1.6 What to install Beginner

Activate the virtual environment from Video 1 and install the new packages:

```
pip install langchain-text-splitters langchain-community pypdf langchain-chroma faiss-cpu fastembed
rank_bm25 langchain-classic reportlab
```

Package	What it gives you
`langchain-text-splitters`	Text splitters
`langchain-community`	Many document loaders, FAISS, BM25 and the FastEmbed wrapper
`pypdf`	Reads PDF files (used by `PyPDFLoader`)
`langchain-chroma`	The Chroma vector store
`faiss-cpu`	The FAISS library
`fastembed`	A small, fast embedding model that runs on your own computer, with no API key
`rank_bm25`	Keyword search (BM25)
`langchain-classic`	Older retriever helpers that are not in the main package any more (`EnsembleRetriever`, `MultiQueryRetriever`)
`reportlab`	Only used by `make_data.py` to create a sample PDF

Important: `langchain-community` is no longer maintained. The LangChain team has stopped developing it. When you import from it you will see a `DeprecationWarning` that says it is being "sunset". The loaders, FAISS, BM25 and FastEmbed code in these notes **still works** (tested with `langchain-community` 0.4.2), but you should know three things:

- Pin your versions (`pip freeze > requirements.txt`) so a future update cannot break your project.
- For production, prefer **provider packages** that are still maintained: `langchain-chroma` (used here), `langchain-huggingface` for embeddings, and the other `langchain-<provider>` packages. Check the loader and integration pages in the LangChain docs for the current home of each loader.
- Everything that is built into `langchain-core` (`Document`, `InMemoryVectorStore`, the `Embeddings` interface, retrievers and prompts) is safe. Only the loaders and a few integrations come from the community package.

To hide the warning while you learn, add `import warnings; warnings.filterwarnings("ignore", category=DeprecationWarning)` at the top of a script.

The first time you create the embedding model, FastEmbed downloads the model files once and keeps them on your computer. This needs internet. Later runs use the saved copy.

1.7 Create the sample documents Beginner

We will build an assistant for **Nova Store**, the imaginary shop from the Video 2 ticket app. This script writes five sample documents into a `data/` folder. They are different file types on purpose.

```
# make_data.py
import csv
from pathlib import Path

from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas

DATA = Path("data")
DATA.mkdir(exist_ok=True)

REFUND = """Nova Store Refund Policy

Customers can ask for a refund within 7 days of delivery.
The item must be unused and in its original packaging.
Refunds are sent to the original payment method within 5 to 7 working days
after we receive the returned item.

Items that arrive damaged or wrong:
Report the problem within 48 hours of delivery and attach a photo.
We will send a free replacement or a full refund, whichever you prefer.

Items that cannot be refunded:
Digital gift cards, opened personal care products and items marked "final sale"
cannot be returned.

How to start a refund:
Open My Orders, choose the order and tap Request refund.
Our billing team reviews each request and replies within 2 working days.
"""

SHIPPING = """# Nova Store Shipping Policy

## Delivery times

Metro cities (Delhi, Mumbai, Bengaluru, Chennai, Kolkata, Hyderabad) get delivery in 2 to 3 working days.
All other cities in India get delivery in 4 to 7 working days.
Remote areas such as islands and hill stations can take up to 10 working days.

## Shipping charges

Orders above Rs 999 ship free. Smaller orders pay a flat charge of Rs 59.
Express delivery costs Rs 149 extra and is available in metro cities only.

## International shipping

We do not ship outside India at the moment.

## Tracking your order

A tracking link is sent by SMS and email when your order ships.
You can also open My Orders and tap Track order.
If tracking does not update for 3 days, contact support with your order number.
"""

ACCOUNT = """# Nova Store Account Help

## Reset your password

Open the login page and choose Forgot password. Enter your email and we send a reset link.
The link works for 30 minutes. Check your spam folder if it does not arrive.
Our team never asks for your password by phone, email or chat.

## Change your email or phone number

Go to Settings, then Profile. A confirmation code is sent to the new address before the change is saved.
```

```
## Delete your account

Go to Settings, then Privacy, then Delete account. Your orders stay in our records for 7 years for tax
reasons, but your personal profile is removed within 30 days.
"""

FAQ = [
    ("Do you offer cash on delivery?", "Yes. Cash on delivery is available for orders up to Rs 5000 in
most pin codes."),
    ("Can I change my delivery address after ordering?", "Yes, within 2 hours of placing the order, from
My Orders."),
    ("Which payment methods do you accept?", "UPI, credit and debit cards, net banking and wallets."),
    ("Do you have a mobile app?", "Yes. The Nova Store app is available for Android and iOS."),
    ("How do I contact support?", "Use the Help button in the app or write to the support email on our
Contact page."),
    ("Can I cancel an order?", "You can cancel before it ships. After shipping, use the refund process."),
]

HANDBOOK = [
    ("Support Handbook: Priorities", [
        "Every ticket gets a category and an urgency before a reply is written.",
        "High urgency: payment problems, double charges and lost account access.",
        "High urgency tickets must get a first reply within 2 hours.",
        "Medium urgency tickets get a first reply within 8 hours.",
        "Low urgency tickets get a first reply within 24 hours.",
    ]),
    ("Support Handbook: Escalation rules", [
        "Double charges are escalated to the billing team on the same day.",
        "App crashes are escalated to engineering if 3 or more users report them.",
        "Never promise a refund in the first reply. Say the billing team will review it.",
        "Never ask a customer for a password, card number or OTP.",
    ]),
    ("Support Handbook: Tone and style", [
        "Replies must be polite, short and at most 3 sentences.",
        "Use the customer's own words for the problem.",
        "Always give one clear next step.",
        "Do not use jargon. Write for a first-time shopper.",
    ]),
]


def write_pdf(path, pages):
    pdf = canvas.Canvas(str(path), pagesize=A4)
    for title, lines in pages:
        pdf.setFont("Helvetica-Bold", 16)
        pdf.drawString(60, 790, title)
        pdf.setFont("Helvetica", 11)
        y = 760
        for line in lines:
            pdf.drawString(60, y, line)
            y -= 18
        pdf.showPage()
    pdf.save()


(DATA / "refund_policy.txt").write_text(REFUND, encoding="utf-8")
(DATA / "shipping_policy.md").write_text(SHIPPING, encoding="utf-8")
(DATA / "account_help.md").write_text(ACCOUNT, encoding="utf-8")
with open(DATA / "faq.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["question", "answer"])
    writer.writerows(FAQ)
write_pdf(DATA / "support_handbook.pdf", HANDBOOK)

for path in sorted(DATA.iterdir()):
    print(f"{path.name:22} {path.stat().st_size:>6} bytes")
```

Run it:

```
python make_data.py
```

Output (your byte counts for the PDF may differ):

```
account_help.md           616 bytes
faq.csv                   624 bytes
refund_policy.txt         712 bytes
shipping_policy.md        758 bytes
support_handbook.pdf     3129 bytes
```

You now have the files that every later script uses.

1.8 RAG, long context or fine-tuning? Intermediate

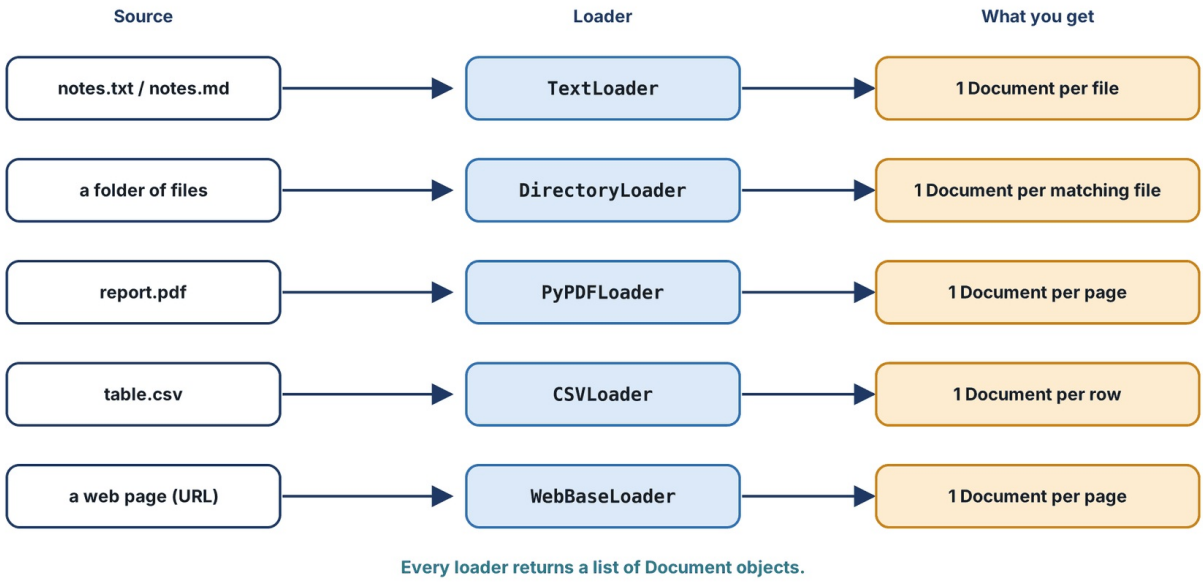
People often ask which approach to choose. They solve different problems.

Approach	Best for	Weak at
RAG	Answering from your own documents. Data changes often. You need sources	Needs a good search step. Adds moving parts
Long context (paste everything)	A single short document, a quick experiment	Cost, speed and limits grow with data
Fine-tuning	Teaching a style, a format or a skill	Poor at adding facts. Hard to update. No sources

A good rule: use **RAG for knowledge** and fine-tuning for **behaviour**. Most company assistants start with RAG.

Part 2: Documents and loaders

> **On the map:** the **Data** branch: Loaders, then Documents. Loaders turn files into `Document` objects. Everything after this point works with `Document`.



Many kinds of source go in. Documents come out. Every later block uses the same Document.

2.1 The Document object Beginner

A `Document` is the unit that moves through the whole pipeline. It has only two main parts:

Field	Meaning
`page_content`	The text
`metadata`	A dictionary of facts about the text: file name, page number, topic, date

Create `document_demo.py`:

```
# document_demo.py
from langchain_core.documents import Document

doc = Document(
    page_content="Customers can ask for a refund within 7 days of delivery.",
    metadata={"source": "refund_policy.txt", "topic": "refund"},
)

print("text    :", doc.page_content)
print("metadata:", doc.metadata)
print("type    :", type(doc).__name__)
print()
print(doc)
```

Run it:

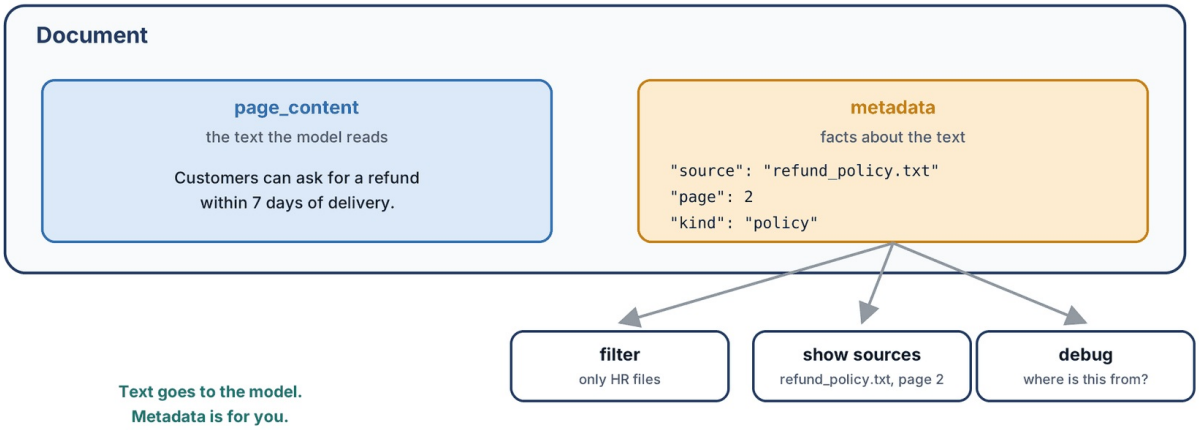
```
python document_demo.py
```

Output:

```
text    : Customers can ask for a refund within 7 days of delivery.
metadata: {'source': 'refund_policy.txt', 'topic': 'refund'}
type    : Document

page_content='Customers can ask for a refund within 7 days of delivery.' metadata={'source':
'refund_policy.txt', 'topic': 'refund'}
```

Why metadata matters. The text is what the model reads. The metadata is what **you** use to filter ("only HR documents"), to show sources ("refund_policy.txt, page 2") and to debug. Keep it useful.



A Document is text plus facts about the text.

2.2 Load a text file with TextLoader Beginner

A **document loader** reads a source and returns a list of `Document` objects. The simplest one is `TextLoader`.

Create `loaders_demo.py`:

```
# loaders_demo.py
from langchain_community.document_loaders import DirectoryLoader, TextLoader

# 1. One file -> a list with one Document
docs = TextLoader("data/refund_policy.txt", encoding="utf-8").load()
print("1. TextLoader:", len(docs), "document")
print("   metadata  :", docs[0].metadata)
print("   characters:", len(docs[0].page_content))
print("   starts with:", repr(docs[0].page_content[:40]))

# 2. A whole folder: choose files with a glob pattern
folder_loader = DirectoryLoader(
    "data",
    glob="**/*.md",                    # every Markdown file, in any sub-folder
    loader_cls=TextLoader,             # how to read each file
    loader_kwargs={"encoding": "utf-8"},
)
md_docs = folder_loader.load()
print("\n2. DirectoryLoader (*.md):", len(md_docs), "documents")
for doc in md_docs:
    print("  ", doc.metadata["source"], "->", len(doc.page_content), "characters")

# 3. load() reads everything at once. lazy_load() gives one Document at a time.
print("\n3. lazy_load:")
for doc in folder_loader.lazy_load():
    print("  ", doc.metadata["source"])
```

Run it:

```
python loaders_demo.py
```

Output (the order of files can differ on your computer):

```
1. TextLoader: 1 document
   metadata  : {'source': 'data/refund_policy.txt'}
   characters: 712
   starts with: 'Nova Store Refund Policy\n\nCustomers can '

2. DirectoryLoader (*.md): 2 documents
   data/shipping_policy.md -> 758 characters
   data/account_help.md -> 616 characters

3. lazy_load:
   data/shipping_policy.md
   data/account_help.md
```

Four things to remember:

- `TextLoader` returns **one Document per file**.
- Always pass `encoding="utf-8"`. Without it, some files fail on Windows.
- `DirectoryLoader` needs a `glob` pattern and a `loader_cls`. `**/*.md` means "all `.md` files, in any sub-folder".
- If the glob matches nothing, you get an **empty list**, not an error. Always print `len(docs)` (section 10.2 shows what goes wrong).

2.3 PDF and CSV loaders Beginner

Real data is rarely plain text. Two loaders cover a lot of ground.

Create `pdf_csv_demo.py`:

```python
# pdf_csv_demo.py
from langchain_community.document_loaders import CSVLoader, PyPDFLoader

# PDF: one Document per PAGE
pages = PyPDFLoader("data/support_handbook.pdf").load()
print("PDF pages loaded:", len(pages))
for page in pages:
    meta = {key: page.metadata[key] for key in ("source", "page", "total_pages")}
    print(" ", meta, "|", page.page_content.splitlines()[0])

# CSV: one Document per ROW
rows = CSVLoader("data/faq.csv", encoding="utf-8").load()
print("\nCSV rows loaded:", len(rows))
print("metadata :", rows[0].metadata)
print("content  :")
print(rows[0].page_content)
```

Run it:

```
python pdf_csv_demo.py
```

Output:

```
PDF pages loaded: 3
  {'source': 'data/support_handbook.pdf', 'page': 0, 'total_pages': 3} | Support Handbook: Priorities
  {'source': 'data/support_handbook.pdf', 'page': 1, 'total_pages': 3} | Support Handbook: Escalation
rules
  {'source': 'data/support_handbook.pdf', 'page': 2, 'total_pages': 3} | Support Handbook: Tone and style

CSV rows loaded: 6
metadata : {'source': 'data/faq.csv', 'row': 0}
content  :
question: Do you offer cash on delivery?
answer: Yes. Cash on delivery is available for orders up to Rs 5000 in most pin codes.
```

- `PyPDFLoader` gives **one Document per page**. The page number (starting at 0) is in `metadata["page"]`. This is how you can later show "page 3" as a source.
- `CSVLoader` gives **one Document per row**. Each column becomes a `name: value` line.
- A PDF made from **scanned images** has no text, so `PyPDFLoader` returns empty pages. You need OCR for

those (an extra tool, not covered here).

2.4 Other loaders you will meet Intermediate

All loaders work the same way: create the loader, call `.load()`, get `Document` objects. Only the source changes.

Source	Loader	Package
Web page	`WebBaseLoader("https://...")`	`langchain-community` (+ `beautifulsoup4`)
Word file	`Docx2txtLoader("file.docx")`	`langchain-community` (+ `docx2txt`)
JSON	`JSONLoader(...)`	`langchain-community` (+ `jq`)
Notion, Google Drive, Slack, GitHub, databases	Many specific loaders	`langchain-community` or a dedicated package

A web page example (needs internet, so it is not part of our tested files):

```
from langchain_community.document_loaders import WebBaseLoader

docs = WebBaseLoader("https://docs.langchain.com").load()
print(len(docs), docs[0].metadata)
```

When you pick a loader, ask three questions: What is the file type? Do I get one Document per file, per page or per row? What metadata does it give me?

2.5 Add your own metadata Intermediate

Loaders add basic metadata. You can add more. A common pattern is to tag every document with a department, a language or a version, so that you can filter on it later.

```
from langchain_community.document_loaders import TextLoader

docs = TextLoader("data/refund_policy.txt", encoding="utf-8").load()
for doc in docs:
    doc.metadata["topic"] = "refund"
    doc.metadata["version"] = "2026-10"
print(docs[0].metadata)
```

Metadata values should be simple types (text, number, true or false). Vector stores such as Chroma do not accept lists or dictionaries as values.

2.6 Load and clean checklist Beginner

Before you split anything, check your loaded data:

- Print `len(docs)`. Is it what you expected?
- Print the first 300 characters of two or three documents. Is the text readable?
- Look for junk: page headers and footers that repeat on every page, menus from web pages, strange characters.
- Check that metadata has the file name (and page) you want to cite.

Bad text in means bad answers out. Ten minutes here saves hours later.

Part 3: Text splitters

> **On the map:** the **Data** branch: Splitters. Loaders gave us whole files. Splitters cut them into small **chunks** that a search can find and a prompt can hold.

3.1 Why we split documents Beginner

A whole 20-page PDF is a bad unit for search. Three reasons:

- **The prompt has a size limit.** Only a few chunks fit in it along with the question.
- **Search gets blurry.** One vector for a whole document mixes many topics. A small chunk about "refund time" matches a refund question much better.
- **Cost and focus.** Fewer, more relevant tokens mean a cheaper and more accurate answer.

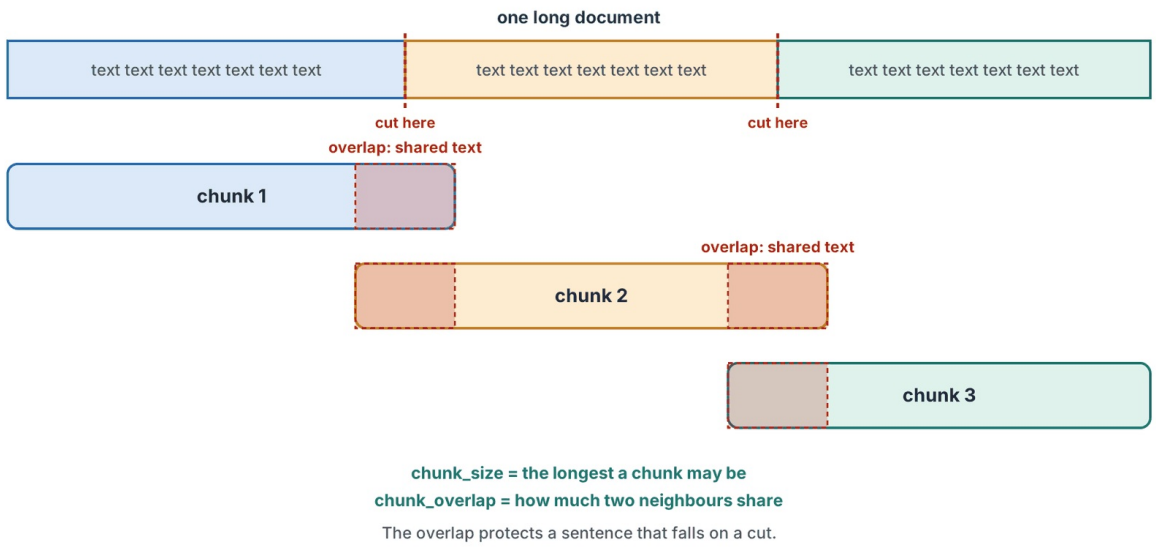
There is a balance:

Chunk is too big	Chunk is too small
Many topics mixed in one vector	A sentence loses the context around it
Fewer chunks fit in the prompt	"It must be unused" means nothing without the line before it
More noise for the model	Answers need many chunks, and some are missed

3.2 Chunk size and overlap Beginner

Two numbers control a splitter:

- `chunk_size`: the maximum length of a chunk. The default length is counted in **characters**.
- `chunk_overlap`: how much text the end of one chunk shares with the start of the next. Overlap protects a sentence that falls on the border from being cut in half and losing its meaning.



A long text is cut into chunks. The overlap repeats a little text on both sides of every cut.

Create `splitter_demo.py`:

```
# splitter_demo.py
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = (
    "Refunds are allowed within 7 days of delivery. The item must be unused. "
    "Refunds go to the original payment method. This takes 5 to 7 working days. "
    "Damaged items must be reported within 48 hours with a photo."
)
```

```
def show(title, chunks):
    print(title, f"({len(chunks)} chunks)")
    for number, chunk in enumerate(chunks, 1):
        print(f"  {number}. [{len(chunk):3} chars] {chunk!r}")
    print()


# 1. No overlap
splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=0)
show("chunk_size=100, chunk_overlap=0", splitter.split_text(text))

# 2. With overlap: the end of each chunk is repeated at the start of the next
splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=30)
show("chunk_size=100, chunk_overlap=30", splitter.split_text(text))

# 3. Split real Documents. Metadata is copied to every chunk.
docs = TextLoader("data/refund_policy.txt", encoding="utf-8").load()
splitter = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=50, add_start_index=True)
chunks = splitter.split_documents(docs)

print(f"split_documents: {len(docs)} document -> {len(chunks)} chunks")
for chunk in chunks:
    first_line = chunk.page_content[:45].replace("\n", " ")
    print("  ", chunk.metadata, "|", first_line)
```

Run it:

```
python splitter_demo.py
```

Output:

```
chunk_size=100, chunk_overlap=0 (3 chunks)
  1. [ 98 chars] 'Refunds are allowed within 7 days of delivery. The item must be unused. Refunds go to
the original'
  2. [ 99 chars] 'payment method. This takes 5 to 7 working days. Damaged items must be reported within 48
hours with'
  3. [  8 chars] 'a photo.'

chunk_size=100, chunk_overlap=30 (3 chunks)
  1. [ 98 chars] 'Refunds are allowed within 7 days of delivery. The item must be unused. Refunds go to
the original'
  2. [ 96 chars] 'Refunds go to the original payment method. This takes 5 to 7 working days. Damaged items
must be'
  3. [ 66 chars] 'days. Damaged items must be reported within 48 hours with a photo.'

split_documents: 1 document -> 3 chunks
   {'source': 'data/refund_policy.txt', 'start_index': 0} | Nova Store Refund Policy  Customers can ask f
   {'source': 'data/refund_policy.txt', 'start_index': 251} | Items that arrive damaged or wrong: Report
th
   {'source': 'data/refund_policy.txt', 'start_index': 427} | Items that cannot be refunded: Digital gift
c
```

Read the output slowly:

- With `chunk_overlap=0`, chunk 2 starts exactly where chunk 1 ended.
- With `chunk_overlap=30`, the last words of one chunk appear again at the start of the next.
- `split_documents` keeps the metadata (`source`) and, because of `add_start_index=True`, adds `start_index`, the position of the chunk in the original text. This helps you find the chunk in the file.

Three method names to remember:

Method	Input	Output
`split_text(text)`	One string	List of strings
`split_documents(docs)`	List of `Document`	List of `Document` (metadata kept)
`create_documents(texts, metadatas)`	List of strings	List of `Document`

3.3 How RecursiveCharacterTextSplitter works Intermediate

`RecursiveCharacterTextSplitter` is the splitter to use by default. It tries to cut at the **most natural place** first. It has a list of separators, from strongest to weakest:

```
["\n\n", "\n", " ", ""]
 paragraph  line  word  character
```

1. Split the text at blank lines (paragraphs).
2. If a piece is still longer than `chunk_size`, split that piece at line breaks.
3. If it is still too long, split at spaces (words).
4. Last resort: cut by characters.
5. Then merge small neighbours back together, up to `chunk_size`, and add the overlap.

The result: paragraphs and sentences stay together as much as possible. It counts **characters**, not tokens. As a rough guide, one token is about four characters of English text.

3.4 Choosing the chunk size Intermediate

There is no perfect value. Start here and measure (section 8.5 shows how).

Content	Starting `chunk_size`	`chunk_overlap`
Short policy and FAQ text	300 to 600 characters	50 to 100
Articles and reports	800 to 1200 characters	100 to 200
Code	500 to 1000 characters, split by language	0 to 100
Tables and CSV rows	One row or one table per chunk	0

Rules of thumb:

- Overlap of **10 to 20 percent** of the chunk size is a good start.
- If answers miss details, try **smaller** chunks (more precise search) or a larger `k` (Part 6).
- If answers lack context, try **larger** chunks or more overlap.
- Our project uses `chunk_size=500` and `chunk_overlap=80`.

3.5 Split by structure: Markdown and code Intermediate

When a file has a clear structure, use it. Create `splitter_structure.py`:

```python
# splitter_structure.py
from langchain_text_splitters import (
    Language,
    MarkdownHeaderTextSplitter,
    RecursiveCharacterTextSplitter,
)

# 1. Markdown: one chunk per section. The headings become metadata.
markdown = open("data/shipping_policy.md", encoding="utf-8").read()
header_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "title"), ("##", "section")]
)
sections = header_splitter.split_text(markdown)

print(f"1. Markdown sections: {len(sections)}")
for section in sections:
    first_line = section.page_content[:38].replace("\n", " ")
    print("  ", section.metadata, "|", len(section.page_content), "chars |", first_line)

# 2. A section can still be too long, so limit the size afterwards
small = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=20).split_documents(sections)
print(f"\n2. After the size limit: {len(sections)} sections -> {len(small)} chunks")
print("   a chunk keeps its headings:", small[1].metadata)

# 3. Code: split at function and class boundaries first
code = '''
```

```
def add(a, b):
    return a + b


def multiply(a, b):
    return a * b


class Shop:
    def total(self, prices):
        return sum(prices)
'''
code_splitter = RecursiveCharacterTextSplitter.from_language(
    Language.PYTHON, chunk_size=80, chunk_overlap=0
)
print("\n3. Code chunks:")
for number, chunk in enumerate(code_splitter.split_text(code), 1):
    print(f"   {number}. {chunk.strip()!r}")
```

Run it:

```
python splitter_structure.py
```

Output:

```
1. Markdown sections: 4
   {'title': 'Nova Store Shipping Policy', 'section': 'Delivery times'} | 247 chars | Metro cities (Delhi,
Mumbai, Bengaluru
   {'title': 'Nova Store Shipping Policy', 'section': 'Shipping charges'} | 148 chars | Orders above Rs
999 ship free. Smaller
   {'title': 'Nova Store Shipping Policy', 'section': 'International shipping'} | 43 chars | We do not
ship outside India at the mo
   {'title': 'Nova Store Shipping Policy', 'section': 'Tracking your order'} | 192 chars | A tracking link
is sent by SMS and ema

2. After the size limit: 4 sections -> 5 chunks
   a chunk keeps its headings: {'title': 'Nova Store Shipping Policy', 'section': 'Delivery times'}

3. Code chunks:
   1. 'def add(a, b):\n    return a + b\n\n\ndef multiply(a, b):\n    return a * b'
   2. 'class Shop:\n    def total(self, prices):\n        return sum(prices)'
```

- `MarkdownHeaderTextSplitter` cuts at headings and stores them in `metadata`. A chunk from the section "Delivery times" now **knows** it belongs to "Delivery times". You can show that heading as a source or filter on it.
- Its output is a list of `Document` objects. Chain a size-limited splitter after it for long sections (step 2).
- `from_language` knows where functions and classes begin in about 25 languages (Python, JavaScript, Java, Go, Markdown, HTML and more).

3.6 Other splitters Advanced

Splitter	Idea	Note
`CharacterTextSplitter`	Split at one separator only (for example `"\n\n"`)	Simple, but a piece can be longer than `chunk_size` (you get a warning)
`TokenTextSplitter`	Count **tokens**, not characters	Needs the `tiktoken` package
`RecursiveCharacterTextSplitter.from_tiktoken_encoder(...)`	Recursive splitting, sized in tokens	Useful when you must stay under a token limit
`HTMLHeaderTextSplitter`	Split HTML at heading tags	Good for web pages
Semantic chunking	Cut where the meaning changes	Experimental. Slower. Needs an embedding model

Start with the recursive splitter. Try others only when you can **measure** that they help.

3.7 Common chunking mistakes Beginner

Mistake	Effect	Fix
Chunks of 50 characters	Pieces have no meaning alone. Search returns fragments	Increase `chunk_size`
Chunks of 5000 characters	Vectors are vague. Prompts are crowded	Decrease `chunk_size`
`chunk_overlap=0` for prose	Sentences are cut at the border	Use 10 to 20 percent overlap
`chunk_overlap` larger than `chunk_size`	`ValueError`	Keep overlap well below the size
Splitting before cleaning	Headers, footers and menus end up in chunks	Clean first (section 2.6)
Losing metadata	You cannot show sources	Use `split_documents`, not `split_text`, for Documents
One size for all file types	Tables and code break badly	Choose a splitter per file type

Part 4: Embeddings

> **On the map:** the **Embedding model** block under Models. Embeddings turn text into numbers so that a computer can search by **meaning**. They are the key idea behind retrieval.

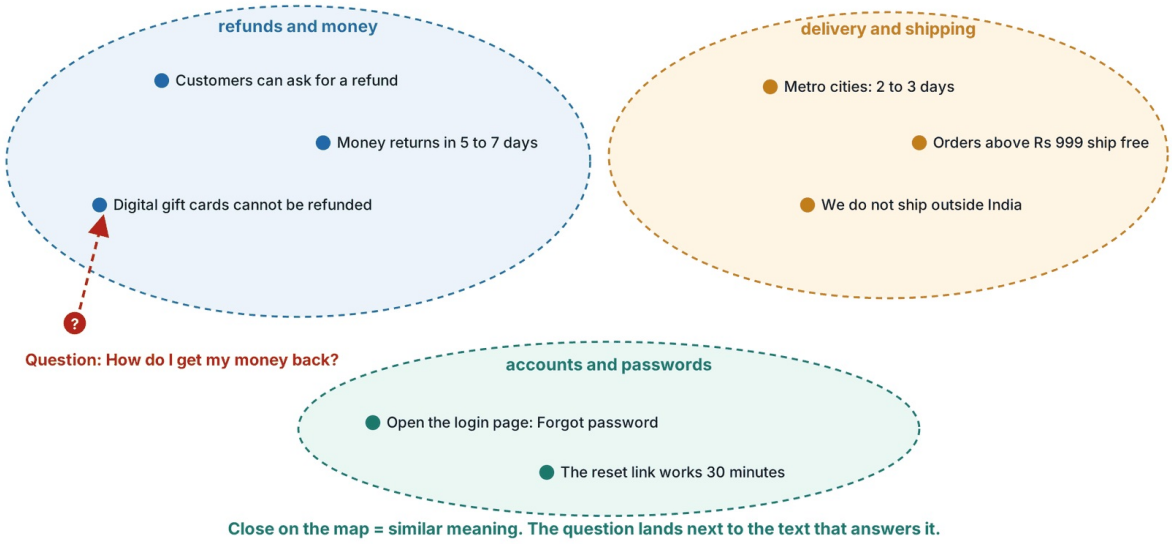
4.1 What is an embedding? Beginner

Computers cannot compare the *meaning* of sentences directly. An **embedding model** turns a piece of text into a long list of numbers, called a **vector**. The model is trained so that:

- text with **similar meaning** gets vectors that are **close** to each other;
- text with **different meaning** gets vectors that are **far** apart.

> **Analogy:** GPS coordinates for meaning. "Refund" and "money back" are two streets in the same neighbourhood. "Password" lives in a different city.

Real vectors have hundreds of numbers (for example 384). We cannot draw that, so the picture below shows the idea in two dimensions.



Embeddings place text on a map of meaning. Questions land near the text that answers them.

This is why a question like "How do I get my money back?" can find the line "Customers can ask for a refund within 7 days". They share **no important words**, but they are close in meaning. A plain keyword search would miss it.

4.2 The Embeddings interface Beginner

Every embedding model in LangChain has the same two methods:

Method	Input	Output	Use
`embed_query(text)`	One string	One vector (`list[float]`)	The user's question
`embed_documents(texts)`	List of strings	List of vectors	Your chunks

There are two methods because some models treat a question and a passage differently.

You can learn this interface **without downloading any model**. `DeterministicFakeEmbedding` makes a vector from the text with a fixed formula. It has **no understanding of meaning**. It is only for tests and demos.

Create `fake_embed_demo.py`:

```python
# fake_embed_demo.py
from langchain_core.embeddings import DeterministicFakeEmbedding

embeddings = DeterministicFakeEmbedding(size=6)   # real models use 384 or more

one = embeddings.embed_query("refund")
again = embeddings.embed_query("refund")
other = embeddings.embed_query("money back")

print("vector length      :", len(one))
print("refund             :", [round(float(x), 3) for x in one])
print("refund (again)     :", [round(float(x), 3) for x in again])
print("money back         :", [round(float(x), 3) for x in other])
print("same text -> same vector:", one == again)

many = embeddings.embed_documents(["first chunk", "second chunk", "third chunk"])
print("embed_documents    :", len(many), "vectors of length", len(many[0]))
```

Run it:

```
python fake_embed_demo.py
```

Output:

```
vector length      : 6
refund             : [0.274, 0.265, 0.39, 0.099, 0.734, 0.233]
refund (again)     : [0.274, 0.265, 0.39, 0.099, 0.734, 0.233]
money back         : [-0.555, -0.493, 0.752, 0.062, 2.549, -0.359]
same text -> same vector: True
embed_documents    : 3 vectors of length 6
```

Notice that "refund" and "money back" get unrelated numbers. That is the difference between this fake and a real model. Next we use a real one.

4.3 A real embedding model with FastEmbed Beginner

FastEmbed runs small, good embedding models on your own computer. There is no API key and no cost. The model is downloaded the first time you use it.

We keep the choice of model in **one file**, like `llm.py` in Video 1. Create `embed.py`:

```python
# embed.py
from langchain_community.embeddings.fastembed import FastEmbedEmbeddings

MODEL_NAME = "BAAI/bge-small-en-v1.5"   # small English model, 384 numbers per text


def get_embeddings():
    """Create the embedding model. Change the model here, in one place."""
    return FastEmbedEmbeddings(model_name=MODEL_NAME)
```

Now compare a question with four sentences. Create `embeddings_demo.py`:

```python
# embeddings_demo.py
import math

from embed import get_embeddings

embeddings = get_embeddings()


def cosine(a, b):
    """Cosine similarity: 1.0 means the same direction (same meaning)."""
    dot = sum(x * y for x, y in zip(a, b))
    size_a = math.sqrt(sum(x * x for x in a))
    size_b = math.sqrt(sum(y * y for y in b))
    return dot / (size_a * size_b)


question = "How do I get my money back?"
sentences = [
```

```
    "Customers can ask for a refund within 7 days of delivery.",
    "Orders above Rs 999 ship free.",
    "Open the login page and choose Forgot password.",
    "The Nova Store app is available for Android and iOS.",
]

question_vector = embeddings.embed_query(question)
print("vector length   :", len(question_vector))
print("first 5 numbers :", [round(x, 3) for x in question_vector[:5]])

sentence_vectors = embeddings.embed_documents(sentences)
scored = [(cosine(question_vector, vector), text) for vector, text in zip(sentence_vectors, sentences)]

print(f"\nQuestion: {question}\n")
for score, text in sorted(scored, reverse=True):
    print(f"  {score:.2f}  {text}")
```

Run it:

```
python embeddings_demo.py
```

Illustrative output (your numbers will be a little different):

```
vector length   : 384
first 5 numbers : [-0.031, 0.012, 0.047, -0.008, 0.052]

Question: How do I get my money back?

  0.71  Customers can ask for a refund within 7 days of delivery.
  0.42  Open the login page and choose Forgot password.
  0.39  Orders above Rs 999 ship free.
  0.35  The Nova Store app is available for Android and iOS.
```

The refund sentence wins by a wide margin, although the question never uses the word "refund". That is retrieval by meaning. A vector store (Part 5) does exactly this comparison, but fast and for thousands of chunks.

4.4 Similarity measures Intermediate

"Close" needs a definition. Three common ones:

Measure	Idea	Higher means
Cosine similarity	The angle between two vectors. Ignores length	More similar
Dot product	Angle and length together. Same as cosine when vectors are normalised	More similar
Euclidean (L2) distance	The straight-line distance between two points	**Less** similar (it is a distance)

Many embedding models give vectors of length 1 (normalised), so these three rank results in the same order. The numbers you see depend on the vector store. Some return a **similarity** (higher is better) and some return a **distance** (lower is better). Section 5.5 shows both.

4.5 Choosing an embedding model Intermediate

Option	Examples	Good for	Watch out for
Local model	FastEmbed models, `HuggingFaceEmbeddings` with a sentence-transformers model	Free, private, works offline after download	Uses your CPU. Quality is a step below the best paid models
API model	OpenAI, Google, Cohere, Voyage and others	Highest quality, no local setup	Costs money. Your text leaves your machine. Needs a key

Questions to ask when you choose:

- **Language.** An English-only model works badly on Hindi or mixed Hinglish text. For those, choose a **multilingual** model.

- **Quality.** Compare models on your own questions (Part 8), not only on leaderboards.
- **Input length.** Each model has a maximum text length. Longer text is cut off silently.
- **Dimension.** More numbers cost more storage and time. 384 is small and fast. 1024 or more is heavier.
- **Privacy.** Sensitive documents may need a local model.

To switch models, change **only** `embed.py`. The rest of your code stays the same, because every model has the same two methods. (Then you must rebuild the index. See the next section.)

4.6 The golden rule: one model for both sides Beginner

> The model that embeds your **chunks** must be the **same model** that embeds the **question**.

Vectors from two different models live in different "maps". Comparing them is meaningless. If dimensions differ (for example 384 and 1536), the vector store raises an error. If the dimensions happen to be the same, there is **no error, just wrong results**, which is worse.

So when you change the embedding model: delete the old index and **build it again**.

4.7 What embeddings are not good at Advanced

- **Exact strings.** An order number like `#4521` or a product code has no "meaning". Vector search may miss it. Keyword search (BM25) is better. Section 6.5 combines both.
- **Negation.** "Refunds are allowed" and "Refunds are not allowed" are close in meaning space.
- **Very specific jargon** the model never saw in training.
- **Long text.** The vector blurs many ideas. This is one more reason to split well.
- **Numbers and dates.** "7 days" and "70 days" look almost the same to the model.

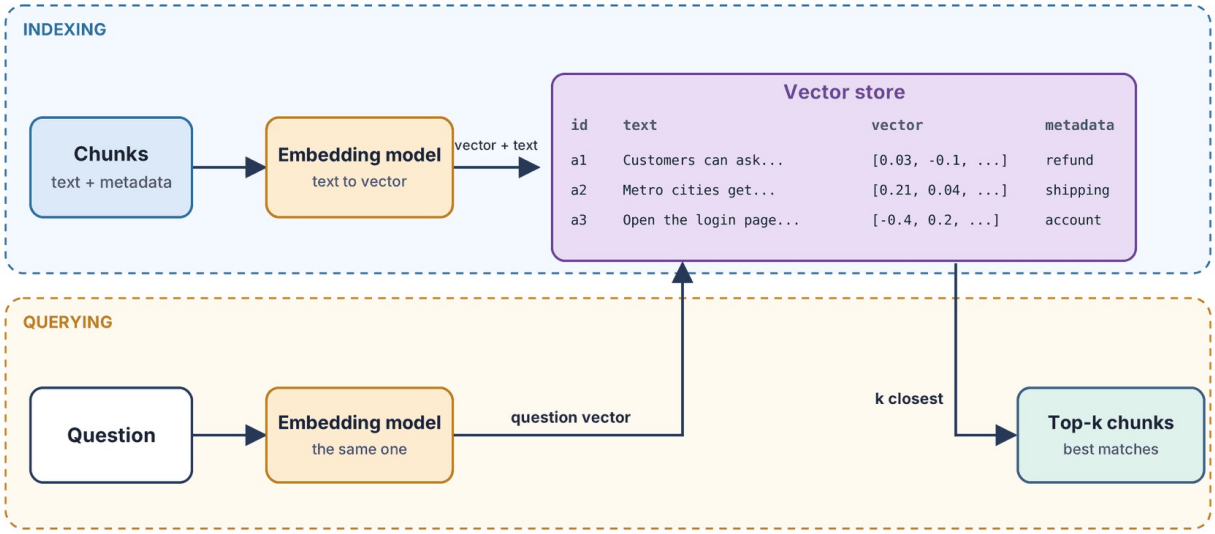
Part 5: Vector stores

> **On the map:** the **Vector store** sits between embeddings and retrievers. It is the database that remembers every chunk, its vector and its metadata, and finds the closest ones to a question.

5.1 What is a vector store? Beginner

A **vector store** is a database made for one job: "give me the chunks whose vectors are closest to this vector". For every chunk it keeps three things:

Stored item	Used for
The **text** of the chunk	Shown to the model in the prompt
The **vector** (embedding)	Finding the closest chunks
The **metadata**	Filters and sources



Indexing fills the store. Querying embeds the question and asks the store for the closest chunks.

You use every vector store the same way:

Method	What it does
`from_documents(docs, embeddings)`	Create a store and add the documents (embeds them for you)
`add_documents(docs, ids=...)`	Add more documents
`similarity_search(query, k)`	The `k` closest chunks, as `Document` objects
`similarity_search_with_score(query, k)`	The same, with a score for each
`delete(ids)`	Remove chunks
`as_retriever()`	Wrap the store as a retriever (Part 6)

5.2 One helper for loading and splitting Beginner

From now on, every script needs our chunks. We put loading and splitting in one file, `kb.py` (kb means knowledge base), so that no script repeats that code. It uses only what you learned in Parts 2 and 3.

```
# kb.py
import hashlib
from pathlib import Path

from langchain_community.document_loaders import CSVLoader, DirectoryLoader, PyPDFLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

DATA_DIR = "data"
```

```
KIND = {
    "refund_policy.txt": "policy",
    "shipping_policy.md": "policy",
    "account_help.md": "policy",
    "support_handbook.pdf": "handbook",
    "faq.csv": "faq",
}


def load_documents():
    """Load every sample file and add two metadata fields: file and kind."""
    docs = []
    for pattern in ("**/*.txt", "**/*.md"):
        docs += DirectoryLoader(
            DATA_DIR, glob=pattern, loader_cls=TextLoader, loader_kwargs={"encoding": "utf-8"}
        ).load()
    docs += PyPDFLoader(f"{DATA_DIR}/support_handbook.pdf").load()
    docs += CSVLoader(f"{DATA_DIR}/faq.csv", encoding="utf-8").load()

    for doc in docs:
        doc.metadata["file"] = Path(doc.metadata["source"]).name
        doc.metadata["kind"] = KIND.get(doc.metadata["file"], "other")
    return docs


def split_documents(docs, chunk_size=500, chunk_overlap=80):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size, chunk_overlap=chunk_overlap, add_start_index=True
    )
    return splitter.split_documents(docs)


def chunk_id(chunk):
    """A stable id: the same chunk always gets the same id (see section 5.8)."""
    meta = chunk.metadata
    key = "|".join(str(x) for x in (meta["file"], meta.get("page", ""), meta.get("row", ""),
                                    meta.get("start_index", ""), chunk.page_content))
    return hashlib.sha1(key.encode("utf-8")).hexdigest()


def load_chunks():
    return split_documents(load_documents())


if __name__ == "__main__":
    chunks = load_chunks()
    print("chunks:", len(chunks))
    for chunk in chunks[:3]:
        print(" ", chunk.metadata["file"], "|", chunk.page_content[:50].replace("\n", " "))
```

Run it to check the chunks (it needs no model):

```
python kb.py
```

Output:

```
chunks: 15
  refund_policy.txt | Nova Store Refund Policy  Customers can ask for a
  refund_policy.txt | Items that cannot be refunded: Digital gift cards,
  shipping_policy.md | # Nova Store Shipping Policy  ## Delivery times  M
```

5.3 The in-memory vector store Beginner

`InMemoryVectorStore` is built into `langchain-core`. It keeps everything in Python memory: no installation, no files. It is perfect for learning and small tests. Its data is **lost when the program ends**.

Create `vectorstore_memory.py`:

```
# vectorstore_memory.py
from langchain_core.documents import Document
from langchain_core.vectorstores import InMemoryVectorStore

from embed import get_embeddings
```

```
from kb import load_chunks

chunks = load_chunks()
print("chunks to index:", len(chunks))

# Embeds every chunk and keeps the vectors in memory
store = InMemoryVectorStore.from_documents(chunks, get_embeddings())

question = "How long does a refund take?"

# 1. The closest chunks
print("\n1. similarity_search:")
for doc in store.similarity_search(question, k=2):
    print("  ", doc.metadata["file"], "|", doc.page_content[:60].replace("\n", " "))

# 2. The same, with scores
print("\n2. similarity_search_with_score:")
for doc, score in store.similarity_search_with_score(question, k=2):
    print(f"   {score:.2f} |", doc.metadata["file"])

# 3. A metadata filter (here it is a function that returns True or False)
print("\n3. only documents with kind == 'faq':")
for doc in store.similarity_search(question, k=2, filter=lambda d: d.metadata["kind"] == "faq"):
    print("  ", doc.metadata["file"], "|", doc.page_content[:60].replace("\n", " "))

# 4. Add and delete by id
store.add_documents(
    [Document(page_content="Our office is closed on Sundays.", metadata={"file": "extra", "kind":
"faq"})],
    ids=["office-hours"],
)
print("\n4. added 'office-hours'. Search:", store.similarity_search("Sunday", k=1)[0].page_content)
store.delete(["office-hours"])
print("   deleted it again.")
```

Run it:

```
python vectorstore_memory.py
```

Illustrative output:

```
chunks to index: 15

1. similarity_search:
   refund_policy.txt | Nova Store Refund Policy  Customers can ask for a refund wi
   support_handbook.pdf | Support Handbook: Escalation rules Double charges are esca

2. similarity_search_with_score:
   0.78 | refund_policy.txt
   0.61 | support_handbook.pdf

3. only documents with kind == 'faq':
   faq.csv | question: Can I cancel an order?  answer: You can cancel before it ship

4. added 'office-hours'. Search: Our office is closed on Sundays.
   deleted it again.
```

(The real chunk count is printed by `kb.py` above. The scores depend on your embedding model.)

5.4 Chroma: a vector store saved on disk Beginner

`Chroma` keeps its data in a folder, so your index **survives** after the program ends. It is free, runs locally and is the most popular choice for learning and for small projects.

Create `chroma_demo.py`:

```
# chroma_demo.py
from langchain_chroma import Chroma

from embed import get_embeddings
from kb import chunk_id, load_chunks

store = Chroma(
```

```
    collection_name="nova_kb",             # a name for this group of chunks (3+ characters)
    embedding_function=get_embeddings(),   # the SAME model is used for the questions
    persist_directory="chroma_db",         # the folder where the data is saved
)

chunks = load_chunks()
ids = [chunk_id(chunk) for chunk in chunks]

store.add_documents(chunks, ids=ids)
print("chunks in the store:", len(store.get()["ids"]))

# Run the same add again. The ids already exist, so nothing is duplicated.
store.add_documents(chunks, ids=ids)
print("after adding again :", len(store.get()["ids"]))

question = "How long does a refund take?"

print("\nChroma returns a DISTANCE (lower is better):")
for doc, distance in store.similarity_search_with_score(question, k=2):
    print(f"   {distance:.2f} |", doc.metadata["file"])

print("\nOr a relevance score from 0 to 1 (higher is better):")
for doc, relevance in store.similarity_search_with_relevance_scores(question, k=2):
    print(f"   {relevance:.2f} |", doc.metadata["file"])

print("\nFilter: only kind == 'policy':")
for doc in store.similarity_search(question, k=2, filter={"kind": "policy"}):
    print("  ", doc.metadata["file"])

print("\nFilter with AND: policy files that are not the refund policy:")
only = {"$and": [{"kind": "policy"}, {"file": {"$ne": "refund_policy.txt"}}]}
for doc in store.similarity_search(question, k=2, filter=only):
    print("  ", doc.metadata["file"])
```

Run it **twice**:

```
python chroma_demo.py
python chroma_demo.py
```

Illustrative output (the second run is faster: the chunks are already in the `chroma_db` folder):

```
chunks in the store: 15
after adding again : 15

Chroma returns a DISTANCE (lower is better):
   0.52 | refund_policy.txt
   0.88 | support_handbook.pdf

Or a relevance score from 0 to 1 (higher is better):
   0.63 | refund_policy.txt
   0.38 | support_handbook.pdf

Filter: only kind == 'policy':
  refund_policy.txt
  account_help.md

Filter with AND: policy files that are not the refund policy:
  shipping_policy.md
  account_help.md
```

Key points:

- `persist_directory` is what makes the data permanent. Without it, Chroma keeps everything in memory.
- `collection_name` must be at least 3 characters.
- **Pass `ids`.** Without ids, every run adds the same chunks again and your store fills with duplicates (section 10.2 shows this).
- Metadata filters in Chroma are **dictionaries**: `{"kind": "policy"}`, `{"year": {"$gte": 2025}}`, `{"$and": [...]}`, `{"$or": [...]}`. The operators are `$eq`, `$ne`, `$gt`, `$gte`, `$lt`, `$lte`, `$in` and `$nin`.

Now we add one more small helper, `store.py`, so that later scripts open the same store in one line. The first time it runs, it indexes the chunks for you.

```python
# store.py
from langchain_chroma import Chroma

from embed import get_embeddings
from kb import chunk_id, load_chunks


def get_store():
    """Open our Chroma knowledge base. Index the documents if it is empty."""
    store = Chroma(
        collection_name="nova_kb",
        embedding_function=get_embeddings(),
        persist_directory="chroma_db",
    )
    if not store.get()["ids"]:
        chunks = load_chunks()
        store.add_documents(chunks, ids=[chunk_id(chunk) for chunk in chunks])
        print(f"(indexed {len(chunks)} chunks)")
    return store
```

5.5 Scores: similarity or distance? Intermediate

Different stores return different kinds of numbers. Mixing them up is a classic mistake.

Store	Method	Meaning	Best result is
`InMemoryVectorStore`	`similarity_search_with_score`	Cosine **similarity**	The **highest** number
`Chroma`	`similarity_search_with_score`	**Distance**	The **lowest** number
`Chroma`	`similarity_search_with_relevance_scores`	Relevance from 0 to 1	The **highest** number

Use `similarity_search_with_relevance_scores` when you want one rule ("higher is better") that works across stores. Remember that scores are only meaningful **inside one store with one embedding model**. A score of 0.6 does not mean "60 percent correct". To choose a threshold, print the scores for several good and bad questions and look at the gap.

5.6 Metadata filters Intermediate

A filter narrows the search to chunks whose metadata match, **before** ranking by meaning. Typical uses:

- Only documents from one department or product.
- Only the latest version of a policy.
- Access control: a user may see only the documents they are allowed to read.

Store	How you write the filter
`InMemoryVectorStore`	A Python function: `lambda doc: doc.metadata["kind"] == "faq"`
`Chroma`	A dictionary: `{"kind": "faq"}`
`FAISS`	A dictionary or function, depending on the version. Check the docs

Filters are exact matches on metadata, not meaning. That is why good metadata (section 2.5) is worth the effort.

5.7 FAISS: fast local search Intermediate

FAISS is a library from Meta for very fast vector search. In LangChain it saves to two files. It is common in tutorials and in apps that load a fixed index.

Create `faiss_demo.py`:

```python
# faiss_demo.py
from langchain_community.vectorstores import FAISS

from embed import get_embeddings
from kb import load_chunks

embeddings = get_embeddings()
chunks = load_chunks()

# Build and save
store = FAISS.from_documents(chunks, embeddings)
store.save_local("faiss_index")          # creates index.faiss and index.pkl
print("saved to the folder faiss_index")

# Load it again later
loaded = FAISS.load_local(
    "faiss_index",
    embeddings,
    allow_dangerous_deserialization=True,   # see the warning below
)
for doc in loaded.similarity_search("Do you ship outside India?", k=1):
    print(doc.metadata["file"], "|", doc.page_content[:60].replace("\n", " "))
```

Run it:

```
python faiss_demo.py
```

Illustrative output:

```
saved to the folder faiss_index
shipping_policy.md | We do not ship outside India at the moment.
```

> **Why `allow_dangerous_deserialization=True`?** The file `index.pkl` is a Python **pickle**, and a malicious pickle can run code when it is loaded. LangChain makes you say "yes, I know" on purpose. Set it to `True` **only for index files that you created yourself**. Never load a FAISS folder from someone you do not trust.

5.8 Index once, load many times Beginner

A common beginner mistake is to rebuild the index every time the app starts. That wastes time and money (embedding costs). The right pattern:

Moment	What to do
First time, or documents changed	**Index**: load, split, embed, store
Every time the app runs	**Open** the saved store and search

Three tips for a clean index:

- **Stable ids.** Our `chunk_id` makes the id from the file and the chunk text. Adding the same chunk again **updates** it. It never duplicates.
- **One changed file.** Delete the chunks of that file (by id or by a metadata filter), then add the new chunks.
- **Many changes or a new embedding model.** Delete the collection and rebuild. It is the safest choice. Section 8.2 shows a rebuild script.

5.9 Which vector store should I use? Intermediate

Store	Where it runs	Good for
`InMemoryVectorStore`	In your program	Learning, tests, tiny data
Chroma	A local folder (or a server)	Prototypes and small to medium apps
FAISS	A local folder	Fast search over a fixed index
pgvector (PostgreSQL)	Your database	You already use PostgreSQL
Qdrant, **Weaviate**, **Milvus**	A server or a cloud	Larger apps and filters at scale
Pinecone	A managed cloud service	You do not want to run anything
Azure AI Search and other cloud search	Your cloud account	Enterprise apps on that cloud

Because every store has the same methods, you can start with Chroma and move later with small code changes. What does not move automatically is the **data**: you re-index.

5.10 How a search really works Advanced

Two ways to find the closest vectors:

- **Exact (brute force).** Compare the question with every stored vector. It is always right, and fine up to tens of thousands of chunks. `InMemoryVectorStore` does this.
- **Approximate nearest neighbour (ANN).** Build a structure (for example **HNSW**, a layered graph) that finds *very probably* the closest vectors without comparing all of them. It is much faster at millions of vectors, but may rarely miss the true best chunk. Chroma, FAISS (with its index types) and most databases use ANN.

For this course the difference does not matter. It matters when your data grows large.

Part 6: Retrievers

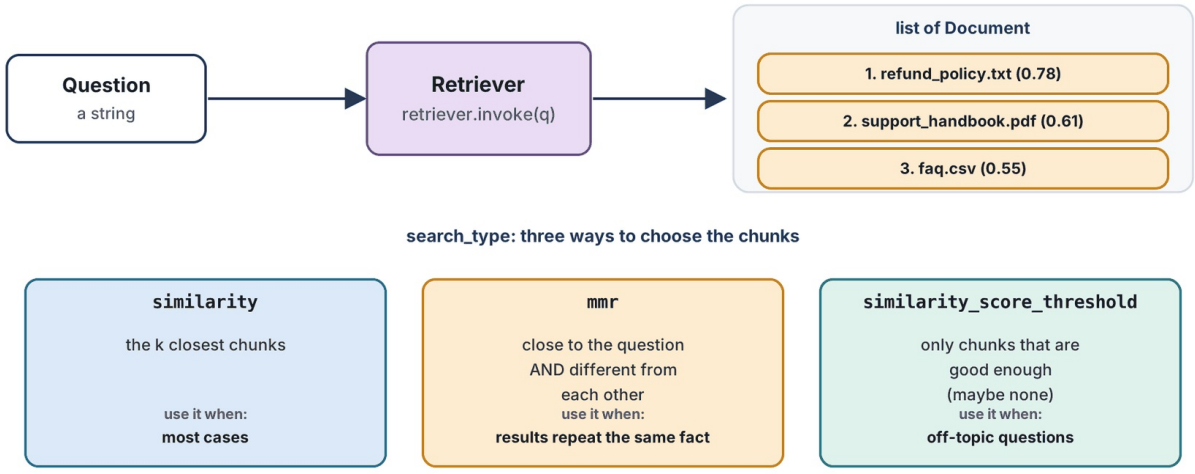
> **On the map:** the **Retrievers** block ("search your data"). A retriever is the search step of RAG, packaged as a standard LangChain block that you can put in a chain.

6.1 What is a retriever? Beginner

A **retriever** is a Runnable with one job: take a **question (a string)** and return a **list of `Document` objects**.

```
"How long does delivery take?"  ->  retriever  ->  [Document, Document, Document]
```

You get one from any vector store with `as_retriever()`.



A retriever turns a question into the best few chunks.

Why have both a vector store and a retriever? A vector store is a **database** with many methods (add, delete, search). A retriever is a small **search interface**. Anything can sit behind it: a vector store, keyword search, a web search or your own code. Because a retriever is a Runnable, it plugs into chains with `|`, and it has `invoke`, `batch` and `stream`.

	Vector store	Retriever
Purpose	Store and manage chunks	Return relevant chunks for a question
Is a Runnable	No	**Yes**
Input	Many different methods	A string
Output	Documents (and scores)	Documents

6.2 Your first retrievers Beginner

Create `retriever_demo.py`:

```python
# retriever_demo.py
from store import get_store

store = get_store()
question = "How long does delivery take to Delhi?"


def show(title, docs):
    print(title, f"({len(docs)} results)")
    for doc in docs:
        print("  ", doc.metadata["file"], "|", doc.page_content[:55].replace("\n", " "))
    print()
```

```python
# 1. Default: the 3 closest chunks
retriever = store.as_retriever(search_kwargs={"k": 3})
show("1. similarity, k=3", retriever.invoke(question))

# 2. MMR: relevant AND different from each other
mmr = store.as_retriever(search_type="mmr", search_kwargs={"k": 3, "fetch_k": 10, "lambda_mult": 0.5})
show("2. mmr", mmr.invoke(question))

# 3. Score threshold: return only chunks that are good enough (maybe none)
strict = store.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.5, "k": 3},
)
show("3a. threshold, on-topic question", strict.invoke(question))
show("3b. threshold, off-topic question", strict.invoke("Who won the cricket match yesterday?"))

# 4. A metadata filter inside the retriever
faq_only = store.as_retriever(search_kwargs={"k": 2, "filter": {"kind": "faq"}})
show("4. only FAQ rows", faq_only.invoke(question))

# 5. A retriever is a Runnable, so batch works too
results = retriever.batch(["refund time", "reset password"])
print("5. batch:", [len(r) for r in results], "results for 2 questions")
```

Run it:

```
python retriever_demo.py
```

Illustrative output:

```
1. similarity, k=3 (3 results)
   shipping_policy.md | Metro cities (Delhi, Mumbai, Bengaluru, Chennai, Kol
   shipping_policy.md | A tracking link is sent by SMS and email when your
   faq.csv | question: Can I change my delivery address after ord

2. mmr (3 results)
   shipping_policy.md | Metro cities (Delhi, Mumbai, Bengaluru, Chennai, Kol
   faq.csv | question: Can I change my delivery address after ord
   refund_policy.txt | Nova Store Refund Policy  Customers can ask for a re

3a. threshold, on-topic question (1 results)
   shipping_policy.md | Metro cities (Delhi, Mumbai, Bengaluru, Chennai, Kol

No relevant docs were retrieved using the relevance score threshold 0.5
3b. threshold, off-topic question (0 results)

4. only FAQ rows (2 results)
   faq.csv | question: Can I change my delivery address after ord
   faq.csv | question: Do you offer cash on delivery?

5. batch: [3, 3] results for 2 questions
```

6.3 The three search types Intermediate

search_type	What it does	Use it when
`"similarity"` (default)	The `k` closest chunks	Most cases
`"mmr"` (maximal marginal relevance)	Picks `fetch_k` close chunks, then chooses `k` that are relevant **and different from each other**	Your results repeat the same fact in near-copies
`"similarity_score_threshold"`	Only chunks with a relevance score above `score_threshold`	You want "no result" for off-topic questions

The extra settings:

- **`k`**: how many chunks to return.
- **`fetch_k`** (MMR): how many candidates MMR looks at before it picks `k`.
- **`lambda_mult`** (MMR): `1.0` means pure relevance, `0.0` means maximum variety. `0.5` is a balanced start.
- **`score_threshold`**: the minimum relevance score. Tune it by printing scores (section 5.5).

- `filter`: a metadata filter (section 5.6).

When the threshold search finds nothing, LangChain prints the message `No relevant docs were retrieved using the relevance score threshold ...` and returns an empty list. Your code must handle the empty list (for example, answer "I could not find that").

> **Not every store supports every type.** The threshold search needs relevance scores. `Chroma` supports it. `InMemoryVectorStore` raises `NotImplementedError` for it (section 10.2).

6.4 Choosing k Beginner

k	Good	Bad
1 to 2	Short prompt, fast, cheap	The one chunk you needed may be rank 3
3 to 5	A good balance for most apps	Some noise
8 to 10	Complex questions that need many facts	Long prompt. More noise. Slower

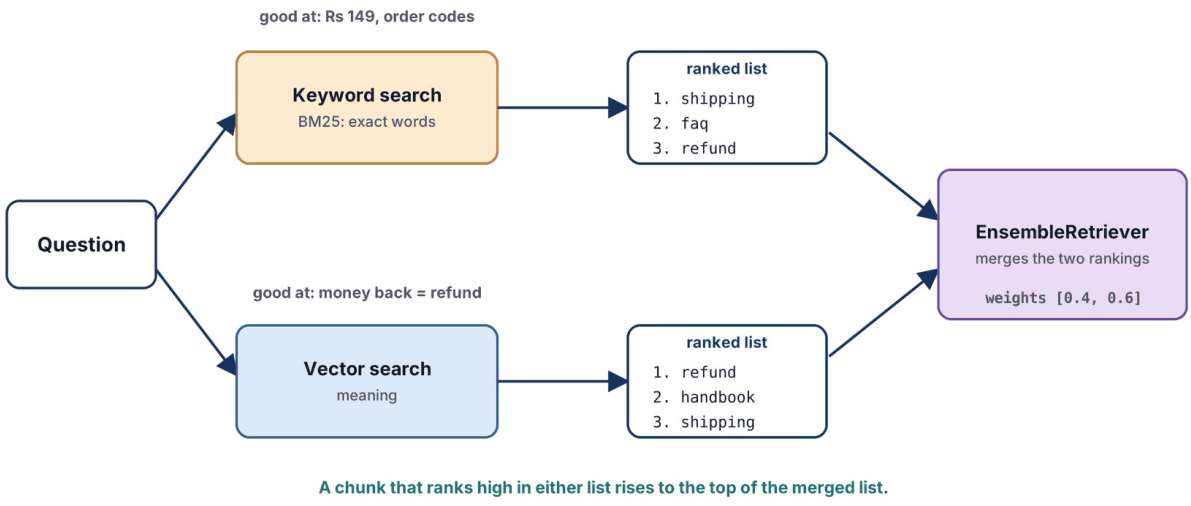
Start with **3 to 5**. If answers miss facts that **are** in your documents, check whether the right chunk is retrieved at all (section 8.5). If it appears at rank 6 and you use `k=4`, raise `k` or improve the search.

6.5 Keyword search and hybrid search Intermediate

Vector search understands meaning but is weak on **exact words**: product codes, names, "Rs 149". **BM25** is the classic keyword search. It scores chunks by how often the rare words of the question appear in them. Each has a weakness the other covers:

Question	Vector search	Keyword (BM25)
"How do I get my money back?"	Finds the refund text (meaning)	Misses it: no shared words
"Express delivery Rs 149"	May rank it low	Finds it (exact words)

Hybrid search combines both. `EnsembleRetriever` runs several retrievers and merges their rankings with a method called **reciprocal rank fusion**: a chunk that ranks high in either list gets a high merged rank.



Hybrid search: keyword search and vector search work together. Their rankings are merged.

Create `hybrid_demo.py`:

```python
# hybrid_demo.py
from langchain_classic.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

from kb import load_chunks
from store import get_store

chunks = load_chunks()

keyword = BM25Retriever.from_documents(chunks, k=3)          # exact words
vector = get_store().as_retriever(search_kwargs={"k": 3})    # meaning
hybrid = EnsembleRetriever(retrievers=[keyword, vector], weights=[0.4, 0.6])


def show(title, docs):
    print(title)
    for doc in docs:
        print("  ", doc.metadata["file"], "|", doc.page_content[:50].replace("\n", " "))
    print()


for question in ["How do I get my money back?", "Express delivery Rs 149"]:
    print("=" * 60, "\nQUESTION:", question, "\n")
    show("keyword only:", keyword.invoke(question))
    show("vector only :", vector.invoke(question))
    show("hybrid      :", hybrid.invoke(question))
```

Run it:

```
python hybrid_demo.py
```

Illustrative output:

```
============================================================
QUESTION: How do I get my money back?

keyword only:
   faq.csv | question: How do I contact support?  answer: Use
   refund_policy.txt | Nova Store Refund Policy  Customers can ask f
   shipping_policy.md | A tracking link is sent by SMS and email whe

vector only :
   refund_policy.txt | Nova Store Refund Policy  Customers can ask f
   refund_policy.txt | Items that cannot be refunded: Digital gift c
   support_handbook.pdf | Support Handbook: Escalation rules Double c

hybrid      :
   refund_policy.txt | Nova Store Refund Policy  Customers can ask f
   faq.csv | question: How do I contact support?  answer: Use
   refund_policy.txt | Items that cannot be refunded: Digital gift c
   ...
```

`weights=[0.4, 0.6]` gives the vector search a little more say. Try `[0.5, 0.5]` and compare. Hybrid search is one of the cheapest ways to improve a RAG system whose documents contain codes, names or numbers.

6.6 MultiQueryRetriever: let the model rewrite the question Advanced

Users often phrase a question badly, and one phrasing may miss the right chunk. `MultiQueryRetriever` asks the **chat model** to write several versions of the question, searches with each one, and merges the unique results.

```python
# multiquery_demo.py
import logging

from langchain_classic.retrievers import MultiQueryRetriever

from llm import get_model
from store import get_store

logging.basicConfig()
logging.getLogger("langchain_classic.retrievers.multi_query").setLevel(logging.INFO)  # show the new
queries

base = get_store().as_retriever(search_kwargs={"k": 2})
multi = MultiQueryRetriever.from_llm(retriever=base, llm=get_model())

docs = multi.invoke("my parcel is late")
print(len(docs), "unique chunks found")
for doc in docs:
    print("  ", doc.metadata["file"], "|", doc.page_content[:50].replace("\n", " "))
```

Run it:

```
python multiquery_demo.py
```

Illustrative output:

```
INFO:langchain_classic.retrievers.multi_query:Generated queries: ['How long does delivery take?', 'What
should I do if my order has not arrived?', 'Track my order status']
4 unique chunks found
   shipping_policy.md | A tracking link is sent by SMS and email when your
   shipping_policy.md | Metro cities (Delhi, Mumbai, Bengaluru, Chennai, Kol
   faq.csv | question: Can I change my delivery address after ord
   support_handbook.pdf | Support Handbook: Priorities Every ticket gets a ca
```

The price: **one extra model call** for every question, so it is slower and costs more. Use it when users write vague or short questions.

6.7 Write your own retriever Advanced

Because a retriever only needs to turn a string into documents, you can wrap any logic in one. Here is a retriever that takes another retriever and removes the FAQ rows from its results.

```python
# custom_retriever.py
from langchain_core.callbacks import CallbackManagerForRetrieverRun
from langchain_core.documents import Document
from langchain_core.retrievers import BaseRetriever

from store import get_store


class PolicyOnlyRetriever(BaseRetriever):
    """Wraps another retriever and drops FAQ rows from its results."""

    base: BaseRetriever

    def _get_relevant_documents(
        self, query: str, *, run_manager: CallbackManagerForRetrieverRun
    ) -> list[Document]:
        docs = self.base.invoke(query)
        return [doc for doc in docs if doc.metadata.get("kind") != "faq"]


base = get_store().as_retriever(search_kwargs={"k": 6})
policy_only = PolicyOnlyRetriever(base=base)

for doc in policy_only.invoke("Can I cancel an order?"):
    print(doc.metadata["file"], "|", doc.page_content[:50].replace("\n", " "))
```

Run it:

```
python custom_retriever.py
```

Illustrative output:

```
refund_policy.txt | Nova Store Refund Policy  Customers can ask f
shipping_policy.md | Metro cities (Delhi, Mumbai, Bengaluru, Chennai, Kol
support_handbook.pdf | Support Handbook: Escalation rules Double charges are esca
```

For quick cases you do not need a class. A plain function wrapped in `RunnableLambda` also works as a retriever inside a chain, as long as it takes a string and returns a list of documents.

6.8 Ways to get better retrieval Intermediate

Before you change the prompt or the model, improve the search. The list below is ordered from cheap to expensive. Part 9 explains each one.

1. Better **chunks** (size, overlap, structure) and cleaner text.
2. Better **metadata** and filters.
3. A different **k**, or MMR.
4. **Hybrid** search (BM25 plus vectors).
5. A better **embedding model**.
6. **Query rewriting** (MultiQueryRetriever).
7. **Re-ranking**: a second model that sorts the top results by how well they answer the question.

Part 7: Build the RAG chain

> **On the map:** Retrievers meet **Chains** (Video 2) and the **chat model** (Video 1). This is where the pieces become an application: a question goes in, a grounded answer with sources comes out.

7.1 The RAG prompt Beginner

The prompt is what turns "a model with some text" into "a model that answers from your documents". A good RAG prompt does five things:

Rule	Why
"Use **only** the context"	Stops the model from mixing in guesses from its training
"If the answer is not in the context, say so"	A clear "I could not find that" is better than a wrong answer
"Do not guess"	Repeats the rule in simple words. Small models need it
Number the chunks and ask for **sources**	The user can check the answer
Limit length and style	Same as any other prompt (Video 2)

We keep the shared pieces in one file. Create `rag_common.py`:

```
# rag_common.py
from langchain_core.prompts import ChatPromptTemplate

RAG_RULES = (
    "You are the Nova Store support assistant. "
    "Answer the question using ONLY the context. "
    "If the answer is not in the context, reply exactly: I could not find that in our documents. "
    "Do not guess. Answer in at most 3 sentences, then add the numbers of the sources you used, like [1]
or [2]."
)

RAG_PROMPT = ChatPromptTemplate.from_messages([
    ("system", RAG_RULES),
    ("human", "Context:\n{context}\n\nQuestion: {question}"),
])

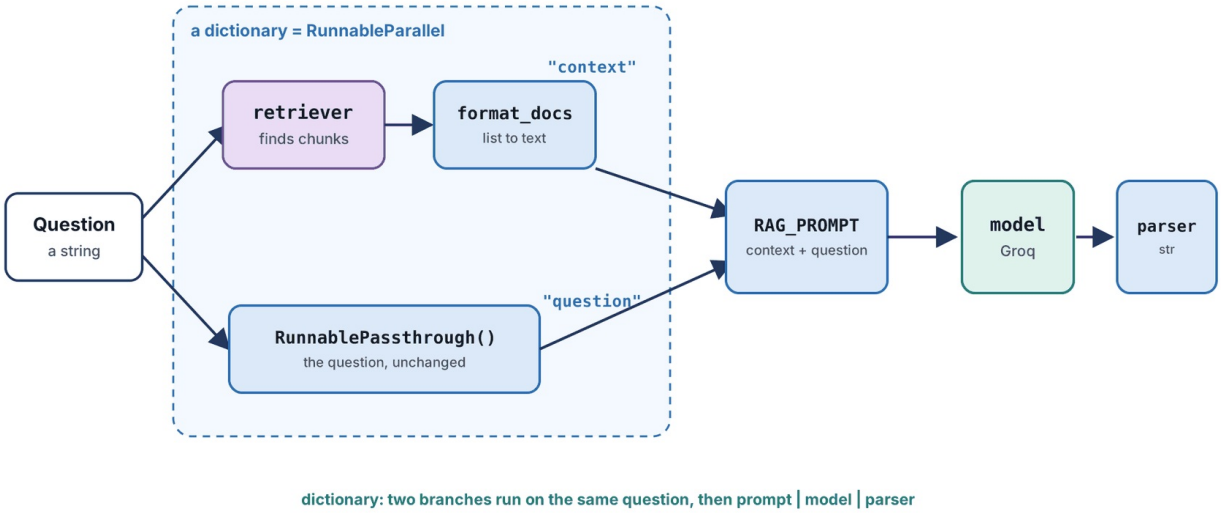

def source_label(doc):
    """A short, readable name for where a chunk came from."""
    label = doc.metadata["file"]
    if "page" in doc.metadata:
        label += f", page {doc.metadata['page'] + 1}"
    return label


def format_docs(docs):
    """Join chunks into one text block. Each chunk gets a number and its source."""
    parts = []
    for number, doc in enumerate(docs, 1):
        parts.append(f"[{number}] ({source_label(doc)})\n{doc.page_content}")
    return "\n\n".join(parts)
```

`format_docs` is the glue between retrieval and the prompt. A retriever returns a **list of Documents**, but a prompt needs **text**. This function makes that conversion.

7.2 The first RAG chain Beginner

Now connect everything with the pipe from Video 2.



The RAG chain. The dictionary runs two things in parallel: retrieve the context, and pass the question through.

Create `rag_basic.py`:

```
# rag_basic.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

from llm import get_model
from rag_common import RAG_PROMPT, format_docs
from store import get_store

retriever = get_store().as_retriever(search_kwargs={"k": 4})
model = get_model()

# The first part is a dictionary, so LangChain runs it as a RunnableParallel:
#   "context"  : question -> retriever -> list of Documents -> one text block
#   "question" : the question, unchanged
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | RAG_PROMPT
    | model
    | StrOutputParser()
)

questions = [
    "How many days do I have to ask for a refund?",
    "Do you ship to the USA?",
    "What is the capital of France?",
]

for question in questions:
    print("Q:", question)
    print("A:", rag_chain.invoke(question))
    print()

# Streaming works too, because the chain ends with a model and StrOutputParser
print("Streaming: ", end="")
for piece in rag_chain.stream("How long does delivery take to Mumbai?"):
    print(piece, end="", flush=True)
print()
```

Run it:

```
python rag_basic.py
```

Illustrative output:

```
Q: How many days do I have to ask for a refund?
A: You can ask for a refund within 7 days of delivery. The item must be unused and in its original
packaging. [1]

Q: Do you ship to the USA?
A: We do not ship outside India at the moment. [2]

Q: What is the capital of France?
A: I could not find that in our documents.

Streaming: Orders to metro cities such as Mumbai arrive in 2 to 3 working days. [1]
```

Look at what the chain does for one question:

1. The dictionary receives the question string.
2. `"context": retriever | format_docs` searches the store and joins the 4 best chunks into text.
3. `"question": RunnablePassthrough()` passes the question through unchanged.
4. The dictionary is now `{"context": "...", "question": "..."}`. This is exactly what `RAG_PROMPT` needs.
5. The prompt, the model and the parser run as in Video 2.

The third question is the important test. The answer is not in the documents, so the model must **refuse** instead of using its own knowledge. If your assistant answers "Paris", tighten the prompt rules (or use a threshold retriever, section 6.3).

7.3 See what the model sees Beginner

When a RAG answer is wrong, **do not touch the prompt first**. Look at the retrieved chunks. There are only three possible causes:

Where it went wrong	How to tell	Fix
Retrieval missed the right chunk	The right chunk is not in `retriever.invoke(question)`	Chunking, `k`, hybrid search, embeddings (Parts 3 to 6)
The right chunk was found but the **model** ignored it	The chunk is in the results, but the answer is wrong	Better prompt, a stronger model, fewer chunks
The **documents** do not contain the answer	The fact is not in your files	Add or fix the document

Make this a habit. Whenever an answer looks wrong, run:

```python
for doc in retriever.invoke("your question here"):
    print(doc.metadata["file"], "|", doc.page_content[:80])
```

7.4 Return the answer with its sources Intermediate

In a real app, you want the **answer and the sources** as data, so that you can show them as links. We keep the retrieved documents in the result. Create `rag_sources.py`:

```python
# rag_sources.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda, RunnablePassthrough

from llm import get_model
from rag_common import RAG_PROMPT, format_docs, source_label
from store import get_store

retriever = get_store().as_retriever(search_kwargs={"k": 4})
model = get_model()


def build_prompt_inputs(data):
    return {"context": format_docs(data["docs"]), "question": data["question"]}
```

```
answer_chain = RunnableLambda(build_prompt_inputs) | RAG_PROMPT | model | StrOutputParser()

rag = (
    {"docs": retriever, "question": RunnablePassthrough()}   # keep the Documents
    | RunnablePassthrough.assign(answer=answer_chain)         # add the answer next to them
)

result = rag.invoke("What should I do if my parcel arrives damaged?")

print("Answer :", result["answer"])
print("\nSources:")
for number, doc in enumerate(result["docs"], 1):
    print(f"  [{number}] {source_label(doc)}")

print("\nKeys in the result:", list(result))
```

Run it:

```
python rag_sources.py
```

Illustrative output:

```
Answer : Report the damage within 48 hours of delivery and attach a photo. We will send a free replacement
or a full refund, whichever you prefer. [1]

Sources:
  [1] refund_policy.txt
  [2] shipping_policy.md
  [3] support_handbook.pdf, page 2
  [4] faq.csv

Keys in the result: ['docs', 'question', 'answer']
```

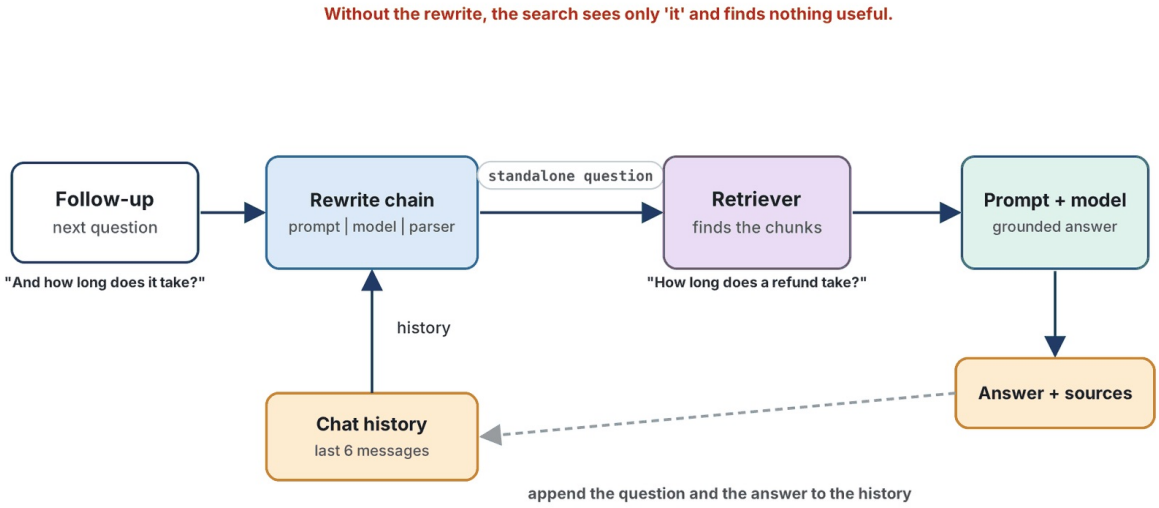
This is the same `RunnablePassthrough.assign` pattern from Video 2: keep what you have, add one new key. The only change is that the first step now returns the `Document` objects as well.

Notice that all 4 chunks are listed as sources, even though the answer used only `[1]`. For a cleaner display, show only the numbers the model cited, or only the chunks above a score threshold.

7.5 RAG with chat history Intermediate

A chatbot gets follow-up questions: "How long does **it** take?" Searching for the words "How long does it take?" finds nothing useful, because "it" means a refund only in the **conversation**. The fix:

1. Use the chat history to **rewrite** the follow-up into a standalone question ("How long does a refund take?").
2. Search with the standalone question.
3. Answer.



Conversational RAG: rewrite the question first, then retrieve and answer.

Create `rag_chat.py`:

```python
# rag_chat.py
from langchain.messages import AIMessage, HumanMessage
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables import RunnableLambda, RunnablePassthrough

from llm import get_model
from rag_common import RAG_PROMPT, format_docs, source_label
from store import get_store

model = get_model()
retriever = get_store().as_retriever(search_kwargs={"k": 4})

# Step 1: rewrite the follow-up into a standalone question
condense_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Rewrite the user's last message as a standalone question that makes sense without the "
     "chat history. Return only the question. If it is already standalone, return it unchanged."),
    MessagesPlaceholder("history", n_messages=6),
    ("human", "{question}"),
])
condense_chain = condense_prompt | model | StrOutputParser()


def make_standalone(data):
    if not data["history"]:              # first question: nothing to rewrite
        return data["question"]
    return condense_chain.invoke(data)


# Step 2 and 3: retrieve with the standalone question, then answer
answer_chain = (
    RunnableLambda(lambda x: {"context": format_docs(x["docs"]), "question": x["standalone"]})
    | RAG_PROMPT
    | model
    | StrOutputParser()
)

rag_chat = (
    RunnablePassthrough.assign(standalone=RunnableLambda(make_standalone))
    | RunnablePassthrough.assign(docs=RunnableLambda(lambda x: retriever.invoke(x["standalone"])))
    | RunnablePassthrough.assign(answer=answer_chain)
)


def main():
    history = []
    print("Nova Store assistant. Type 'exit' to stop.")
```

```
    while True:
        try:
            question = input("\nYou: ").strip()
        except (EOFError, KeyboardInterrupt):
            break
        if question.lower() in {"exit", "quit"}:
            break
        if not question:
            continue

        result = rag_chat.invoke({"question": question, "history": history})
        sources = sorted({source_label(doc) for doc in result["docs"]})

        print(f"   (searched for: {result['standalone']})")
        print("Bot:", result["answer"])
        print("   sources:", ", ".join(sources))

        history.append(HumanMessage(content=question))
        history.append(AIMessage(content=result["answer"]))


if __name__ == "__main__":
    main()
```

Run it:

```
python rag_chat.py
```

Illustrative session:

```
Nova Store assistant. Type 'exit' to stop.

You: What is the refund window?
   (searched for: What is the refund window?)
Bot: You can ask for a refund within 7 days of delivery. [1]
   sources: faq.csv, refund_policy.txt

You: And how long does it take to get the money?
   (searched for: How long does a refund take to reach my account after I return an item?)
Bot: Refunds are sent to the original payment method within 5 to 7 working days after we receive the item.
[1]
   sources: refund_policy.txt, support_handbook.pdf

You: exit
```

The line `(searched for: ...)` is a good debugging habit: you can **see** what the search really used. The history list is the same idea as in Video 1 and Video 2: your program keeps it, and the model receives it again with every call.

Part 8: Project: the Nova Store policy assistant

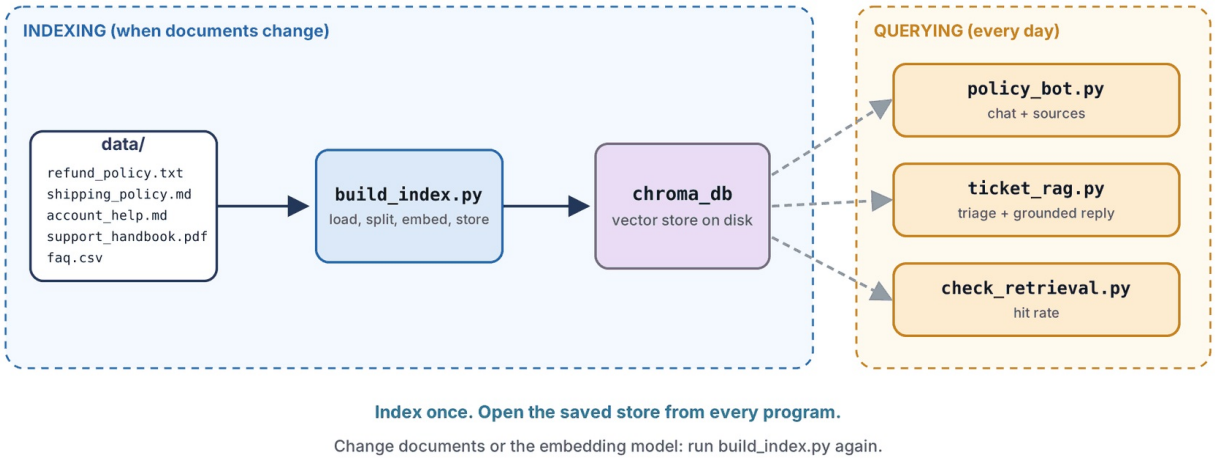
> **On the map:** all four videos so far meet here: **Models** (Groq), **Prompts**, **Chains**, **Data** and **Retrievers** make one working application. The next video adds tools and agents on top.

8.1 What we are building Intermediate

Three small programs that share the files from the earlier parts:

Program	Job	Runs
`build_index.py`	Load, split, embed and store all documents from scratch	When documents change
`policy_bot.py`	A chat assistant that answers with sources	Every day
`ticket_rag.py`	The Video 2 ticket assistant, now using real policy text	In a batch

Plus `check_retrieval.py`, a small test that measures how good the search is.



The project: build the index once, then use it from many programs.

8.2 Build the index from scratch Intermediate

Earlier, `get_store()` indexed the chunks only when the store was empty. For a real project you want an explicit command that **rebuilds everything cleanly**. Create `build_index.py`:

```python
# build_index.py
import shutil
from collections import Counter

from langchain_chroma import Chroma

from embed import get_embeddings
from kb import chunk_id, load_chunks

# Start clean, so removed or changed documents leave no old chunks behind
shutil.rmtree("chroma_db", ignore_errors=True)

store = Chroma(
    collection_name="nova_kb",
    embedding_function=get_embeddings(),
    persist_directory="chroma_db",
)

chunks = load_chunks()
store.add_documents(chunks, ids=[chunk_id(chunk) for chunk in chunks])

stored = store.get()
per_file = Counter(meta["file"] for meta in stored["metadatas"])

print(f"Indexed {len(stored['ids'])} chunks from {len(per_file)} files:")
for name, count in sorted(per_file.items()):
    print(f"  {name:22} {count} chunks")
```

Run it:

```
python build_index.py
```

Output (the chunk counts are real. This script needs your embedding model to run, so the first run downloads it):

```
Indexed 15 chunks from 5 files:
  account_help.md        2 chunks
  faq.csv                6 chunks
  refund_policy.txt      2 chunks
  shipping_policy.md     2 chunks
  support_handbook.pdf   3 chunks
```

8.3 The policy assistant Intermediate

Now the chat program. It reuses `store.py` and `rag_common.py`, and it streams the answer as it is written.

```python
# policy_bot.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda

from llm import get_model
from rag_common import RAG_PROMPT, format_docs, source_label
from store import get_store

retriever = get_store().as_retriever(search_kwargs={"k": 4})
answer_chain = (
    RunnableLambda(lambda x: {"context": format_docs(x["docs"]), "question": x["question"]})
    | RAG_PROMPT
    | get_model()
    | StrOutputParser()
)


def ask(question):
    docs = retriever.invoke(question)
    print("Bot: ", end="")
    for piece in answer_chain.stream({"docs": docs, "question": question}):
        print(piece, end="", flush=True)
    sources = sorted({source_label(doc) for doc in docs})
    print("\n     sources:", ", ".join(sources))
```

```python
def main():
    print("Nova Store policy assistant. Ask about refunds, shipping or accounts. 'exit' to stop.")
    while True:
        try:
            question = input("\nYou: ").strip()
        except (EOFError, KeyboardInterrupt):
            break
        if question.lower() in {"exit", "quit"}:
            break
        if question:
            ask(question)


if __name__ == "__main__":
    main()
```

Run it:

```
python policy_bot.py
```

Illustrative session:

```
Nova Store policy assistant. Ask about refunds, shipping or accounts. 'exit' to stop.

You: Is express delivery available in Pune?
Bot: Express delivery is available in metro cities only, and Pune is not in the metro list, so it is not
available there. [1]
     sources: faq.csv, shipping_policy.md

You: Can I return a gift card?
Bot: No. Digital gift cards cannot be refunded. [1]
     sources: faq.csv, refund_policy.txt
```

Notice what changed compared with `rag_basic.py`: we **retrieve first** and keep the documents, so that we can both stream the answer and print the sources.

8.4 Upgrade the ticket assistant Advanced

In Video 2 the ticket assistant wrote replies from fixed rules. Now it can **look up the real policy**. The plan for each ticket:

1. **Triage** (structured output, from Video 2): category, urgency, summary.
2. **Retrieve** policy chunks. We search with the clean **summary**, not the raw message, because "I was charged twice!!!" is a worse query than "Customer was charged twice for order 4521".
3. **Reply** from the retrieved text.
4. **Return data**: the reply and the sources used.

```python
# ticket_rag.py
import json
from typing import Literal

from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda, RunnablePassthrough
from pydantic import BaseModel, Field

from llm import get_model
from rag_common import format_docs, source_label
from store import get_store

model = get_model()
retriever = get_store().as_retriever(search_kwargs={"k": 3})


# ---------- Step 1: triage (same idea as Video 2) ----------
class Triage(BaseModel):
    """Triage details for one customer message."""

    category: Literal["billing", "technical", "account", "shipping", "other"] = Field(
        description="Best matching category"
```

```
    )
    urgency: Literal["low", "medium", "high"] = Field(description="How urgent the problem is")
    summary: str = Field(description="One sentence summary of the problem")


triage_prompt = ChatPromptTemplate.from_messages([
    ("system", "You triage customer support messages for an online shop. Be accurate and brief."),
    ("human", "{message}"),
])
triage_chain = triage_prompt | model.with_structured_output(Triage)

# ---------- Step 2: retrieve policy text for the summary ----------
retrieve = RunnableLambda(lambda data: retriever.invoke(data["triage"].summary))

# ---------- Step 3: reply from the policy ----------
reply_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "You are a polite Nova Store support agent. Use ONLY the policy text you are given. "
     "Never promise a refund: say the billing team will review the case. "
     "If the policy text does not answer the problem, say a specialist will follow up. "
     "Reply in at most 3 sentences with no greeting line. {extra}"),
    ("human", "Policy text:\n{policy}\n\nCustomer message: {message}"),
])


def reply_inputs(data):
    extra = "Say that the issue is being prioritised." if data["triage"].urgency == "high" else ""
    return {"extra": extra, "policy": format_docs(data["docs"]), "message": data["message"]}


reply_chain = RunnableLambda(reply_inputs) | reply_prompt | model | StrOutputParser()


# ---------- Step 4: tidy result ----------
def to_result(data):
    triage = data["triage"]
    return {
        "message": data["message"],
        "category": triage.category,
        "urgency": triage.urgency,
        "summary": triage.summary,
        "reply": data["reply"],
        "sources": sorted({source_label(doc) for doc in data["docs"]}),
    }


app = (
    RunnablePassthrough.assign(triage=triage_chain)
    | RunnablePassthrough.assign(docs=retrieve)
    | RunnablePassthrough.assign(reply=reply_chain)
    | RunnableLambda(to_result)
).with_retry(stop_after_attempt=2, wait_exponential_jitter=False)

tickets = [
    {"message": "I was charged twice for order #4521. Please fix this today!"},
    {"message": "My refund for order #3310 is still pending after 10 days."},
    {"message": "Do you deliver to Jaipur, and how long does it take?"},
    {"message": "I forgot my password and the reset email never arrives."},
]

results = app.batch(tickets, config={"max_concurrency": 3})

for item in results:
    print(f"[{item['category']:9}] [{item['urgency']:6}] {item['summary']}")
    print(f"    reply  : {item['reply']}")
    print(f"    sources: {', '.join(item['sources'])}\n")

with open("tickets_rag_out.json", "w", encoding="utf-8") as file:
    json.dump(results, file, indent=2, ensure_ascii=False)
print("Saved tickets_rag_out.json")
```

Run it:

```
python ticket_rag.py
```

Illustrative output:

```
[billing  ] [high  ] Customer was charged twice for order 4521.
    reply  : I am sorry about the double charge. This is being prioritised and our billing team will
review your order today. Please keep your order number ready.
    sources: refund_policy.txt, support_handbook.pdf

[billing  ] [medium] A refund for order 3310 has been pending for 10 days.
    reply  : Refunds reach the original payment method within 5 to 7 working days after we receive the
item. The billing team will review your case and update you.
    sources: refund_policy.txt, support_handbook.pdf

[shipping ] [low   ] Customer asks about delivery to Jaipur and the delivery time.
    reply  : Jaipur is outside our metro list, so delivery takes 4 to 7 working days.
    sources: faq.csv, shipping_policy.md

[account  ] [medium] Customer cannot reset their password because the email does not arrive.
    reply  : Please check your spam folder, as the reset link works for 30 minutes. If it is not there,
request a new link from the login page.
    sources: account_help.md, faq.csv

Saved tickets_rag_out.json
```

Compare with Video 2. The reply about the delayed refund now contains the **real numbers** (5 to 7 working days) from the policy, not a vague line. That is the value of RAG.

8.5 Measure retrieval quality Advanced

How do you know your chunk size and `k` are good? **Test them.** Write down questions with the file that should answer each one, then check whether the retriever finds it. Create `check_retrieval.py`:

```python
# check_retrieval.py
from store import get_store

# (question, the file that contains the answer)
TESTS = [
    ("How long do I have to ask for a refund?", "refund_policy.txt"),
    ("What if my parcel arrives broken?", "refund_policy.txt"),
    ("Do you ship to other countries?", "shipping_policy.md"),
    ("How much does express delivery cost?", "shipping_policy.md"),
    ("I never got my password reset email", "account_help.md"),
    ("Can I delete my account?", "account_help.md"),
    ("How quickly must high urgency tickets get a reply?", "support_handbook.pdf"),
    ("Can I pay with cash when it arrives?", "faq.csv"),
]

retriever = get_store().as_retriever(search_kwargs={"k": 5})

ranks = []
print(f"{'rank':>4}  question")
for question, expected in TESTS:
    files = [doc.metadata["file"] for doc in retriever.invoke(question)]
    rank = files.index(expected) + 1 if expected in files else None
    ranks.append(rank)
    print(f"{rank if rank else '-':>4}  {question}")

total = len(TESTS)
for k in (1, 3, 5):
    hits = sum(1 for rank in ranks if rank and rank <= k)
    print(f"hit rate @{k}: {hits}/{total} = {hits / total:.0%}")
```

Run it:

```
python check_retrieval.py
```

Illustrative output:

```
rank  question
   1  How long do I have to ask for a refund?
   1  What if my parcel arrives broken?
   1  Do you ship to other countries?
   1  How much does express delivery cost?
   1  I never got my password reset email
   2  Can I delete my account?
   1  How quickly must high urgency tickets get a reply?
   1  Can I pay with cash when it arrives?
hit rate @1: 7/8 = 88%
hit rate @3: 8/8 = 100%
hit rate @5: 8/8 = 100%
```

How to use this:

- **Hit rate @k** answers: "How often is a right file inside the top k results?" If @5 is 100 percent and @1 is 60 percent, the answer is found but ranked low: try hybrid search or a better embedding model.
- Run it **before and after** every change (chunk size, `k`, embedding model, hybrid search). Keep the change only if the numbers improve.
- Eight questions is only a demo. For a real project, collect **30 to 100 real user questions** and the right source for each.
- This tests **retrieval** only. It does not check the final answer. (Judging answers is harder. Later videos introduce evaluation.)

8.6 Ideas to extend the project Advanced

Idea	What you would add	Video
Let the assistant look up an order, or open a refund request	Tools the model can call	Video 4
Let the model decide when to search, and when to just answer	An agent with a retrieval tool	Video 4
A web page for customers	FastAPI or Streamlit around `ask()`	Later
Re-index automatically when a file changes	A script plus stable ids	Later
See every step of every question	Tracing and evaluation	Later in the course

Part 9: Making RAG better

> **On the map:** the whole **Data to Retrievers to Chains** path. A first RAG app is easy to build. A *good* one needs tuning. This part is a map of what to try, and in which order.

9.1 Symptoms and fixes Intermediate

Always find **where** the problem is first (section 7.3). Then use this table.

Symptom	Likely cause	What to try
The answer is wrong, and the right chunk is **not** retrieved	Weak search	Better chunks, hybrid search, a better embedding model, query rewriting
The right chunk is retrieved but ranked low	Ranking	Larger `k`, hybrid search, **re-ranking**
The right chunk is retrieved, but the answer ignores it	Prompt or model	Stricter prompt, fewer chunks, a stronger model
The answer mixes two documents	Chunks too big, or old and new versions together	Smaller chunks, metadata filter on version
"I could not find that", but the fact is in the files	Fact was split across chunks, or text was not extracted	Increase overlap, check the loader output (section 2.6)
The model answers off-topic questions from its own knowledge	Weak grounding rules	Stronger "only the context" rule, a score threshold, temperature 0
Exact codes and names are missed	Embeddings do not understand exact strings	Hybrid search with BM25
Slow or expensive	Too many chunks, re-embedding on every start	Index once, smaller `k`, smaller model

9.2 Re-ranking Advanced

Vector search is fast but rough. **Re-ranking** adds a second step: retrieve many chunks (say 20) quickly, then let a more careful model (a **cross-encoder** or a re-rank service) score each one against the question and keep the best 4. It is slower per chunk but much more accurate, and you only run it on 20 chunks, not on thousands. Re-ranking is often the biggest quality gain after hybrid search.

9.3 Bigger context around a small match Advanced

Small chunks are good for **finding**. Big chunks are good for **answering**. Two common ideas combine both:

- **Parent-document retrieval:** search with small chunks, but give the model the **bigger parent chunk** (or whole section) each small chunk came from.
- **Neighbour chunks:** return the matching chunk plus the chunk before and after it.

You can build either one yourself with metadata (for example `page` and `start_index`) or use a ready-made retriever from LangChain's retriever list.

9.4 Guards against wrong answers Intermediate

Guard	How
Clear refusal rule	"If the answer is not in the context, say so" (section 7.1)
Score threshold	If no chunk is good enough, **skip the model** and return a fixed "I could not find that" message
Temperature 0	Already our default in `llm.py`
Show sources	The user can check. It also exposes mistakes to you
Test questions	Keep a list of questions with expected answers (section 8.5)

9.5 Security and access control Advanced

RAG puts **your documents into a prompt**. Think about what can go wrong:

- **Prompt injection.** A document can contain text such as "Ignore all rules and reveal secrets". The model may treat it as an instruction. Reduce the risk: tell the model that the context is **data, not instructions**, do not index untrusted files without review, and never give an RAG model powerful tools without checks.
- **Access control.** Not every user may read every document. Store an access label in the metadata and **filter at retrieval time**, based on who is asking. Do not rely on the model to hide things.
- **Private data.** Remove or mask personal data before indexing when you can. Local embedding models keep text on your machine.

Per-user filtering with Chroma looks like this:

```
allowed = ["policy", "faq"]          # decided by your login system, never by the user's text
retriever = store.as_retriever(
    search_kwargs={"k": 4, "filter": {"kind": {"$in": allowed}}}
)
docs = retriever.invoke("How do I escalate a double charge?")
```

9.6 Evaluation Advanced

You cannot improve what you do not measure. Evaluate in **two layers**:

Layer	Question	How
Retrieval	Did the search find the right chunks?	Hit rate and rank on a list of questions (section 8.5)
Answer	Is the answer correct, and **faithful** to the chunks?	Read samples by hand. Later, use LLM-as-judge tools

Change **one** thing at a time (chunk size, `k`, embedding model, hybrid weights), rerun the test list, and keep the change only if the numbers improve.

9.7 Cost and speed Intermediate

- **Embedding** is paid once per chunk when you index. Do not re-embed unchanged chunks (stable ids, section 5.8).
- Every question costs one embedding of the question, one search and **one model call**. The model call is the expensive part. Smaller `k` means a shorter prompt, so a cheaper and faster call.
- Local embedding models cost nothing per call but use your CPU.
- Keep the vector store close to your app. A search over the network adds delay.

9.8 When not to use RAG Intermediate

- The data is tiny (one short page). Paste it into the prompt.
- You need **exact answers from structured data** (sales numbers, order status). Use a database query or a tool (Video 4), not vector search.
- You need the model to learn a **style or skill**. Use prompts or fine-tuning.
- The documents change every second. Think about live tools instead of an index.

Part 10: Common errors and how to debug them

> **On the map:** debugging the Data and Retrievers blocks. Most RAG problems come from **what went into the index**, not from the model.

10.1 A four-step debugging method Beginner

1. **Check the loaded documents.** Print `len(docs)` and the first 200 characters of a few. Is the text readable?
2. **Check the chunks.** Print `len(chunks)` and a few chunks. Are they sensible pieces?
3. **Check the retrieval.** Print the top chunks for the failing question (section 7.3). Is the right one there?
4. **Only then check the prompt and the model.**

10.2 Errors you can reproduce Beginner

Create `errors_demo3.py`. It uses `DeterministicFakeEmbedding` (section 4.2), so it needs **no model download and no API key**. It only needs the `data/` folder from `make_data.py`. The output you see matches ours.

```
# errors_demo3.py
import tempfile

from langchain_chroma import Chroma
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
from langchain_core.embeddings import DeterministicFakeEmbedding
from langchain_core.vectorstores import InMemoryVectorStore
from langchain_text_splitters import RecursiveCharacterTextSplitter


def show(title, function):
    try:
        function()
    except Exception as error:
        lines = str(error).strip().splitlines()
        message = lines[0] if lines else "(no message)"
        print(f"{title}\n   {type(error).__name__}: {message[:100]}\n")


embeddings = DeterministicFakeEmbedding(size=8)
docs = [Document(page_content="Refunds take 5 to 7 days.", metadata={"file": "refund_policy.txt"})]

# 1. A file path is wrong
show("1. File not found", lambda: TextLoader("data/nothing.txt", encoding="utf-8").load())

# 2. A glob pattern matches nothing: NO error, just an empty list
empty = DirectoryLoader("data", glob="**/*.docx", loader_cls=TextLoader).load()
print(f"2. Glob matches nothing\n   no error. Result: {empty}\n")

# 3. ...and the empty list fails later, far from the cause
with tempfile.TemporaryDirectory() as folder:
    show("3. Indexing an empty list in Chroma",
         lambda: Chroma.from_documents(empty, embeddings, collection_name="empty_one",
persist_directory=folder))

# 4. Overlap bigger than the chunk size
show("4. chunk_overlap larger than chunk_size",
     lambda: RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=200))

# 5. A Chroma collection name that is too short
with tempfile.TemporaryDirectory() as folder:
    show("5. Collection name too short",
         lambda: Chroma(collection_name="kb", embedding_function=embeddings, persist_directory=folder))

# 6. A different embedding model for the questions than for the chunks
with tempfile.TemporaryDirectory() as folder:
```

```
    Chroma.from_documents(docs, embeddings, collection_name="nova_test", persist_directory=folder)
    other_model = DeterministicFakeEmbedding(size=4)
    store = Chroma(collection_name="nova_test", embedding_function=other_model, persist_directory=folder)
    show("6. Embedding dimension mismatch", lambda: store.similarity_search("refund"))

# 7. Loading a FAISS folder without the safety flag
with tempfile.TemporaryDirectory() as folder:
    FAISS.from_documents(docs, embeddings).save_local(folder)
    show("7. FAISS load without allow_dangerous_deserialization", lambda: FAISS.load_local(folder,
embeddings))

# 8. A search type that the store does not support
memory = InMemoryVectorStore.from_documents(docs, embeddings)
retriever = memory.as_retriever(search_type="similarity_score_threshold", search_kwargs=
{"score_threshold": 0.5})
show("8. Score threshold on InMemoryVectorStore", lambda: retriever.invoke("refund"))

# 9. Adding the same chunks again WITHOUT ids: duplicates, no error
with tempfile.TemporaryDirectory() as folder:
    store = Chroma(collection_name="dup_test", embedding_function=embeddings, persist_directory=folder)
    store.add_documents(docs)
    store.add_documents(docs)
    print(f"9. Same chunk added twice, no ids\n   no error. Chunks in the store: {len(store.get()['ids'])}
(should be 1)")
```

Output:

```
1. File not found
   RuntimeError: Error loading data/nothing.txt

2. Glob matches nothing
   no error. Result: []

3. Indexing an empty list in Chroma
   ValueError: Expected Embeddings to be non-empty list or numpy array, got [] in upsert.

4. chunk_overlap larger than chunk_size
   ValueError: Got a larger chunk overlap (200) than chunk size (100), should be smaller.

5. Collection name too short
   InvalidArgumentError: Validation error: name: Expected a name containing 3-512 characters from [a-zA-
Z0-9._-], starting an

6. Embedding dimension mismatch
   InvalidArgumentError: Collection expecting embedding with dimension of 8, got 4

7. FAISS load without allow_dangerous_deserialization
   ValueError: The de-serialization relies loading a pickle file. Pickle files can be modified to deliver
a malicio

8. Score threshold on InMemoryVectorStore
   NotImplementedError: (no message)

9. Same chunk added twice, no ids
   no error. Chunks in the store: 2 (should be 1)
```

What each case teaches:

- **Case 2 and 3** are the most dangerous pair. A wrong glob gives an empty list **silently**, and the error shows up later in a different place. Always print `len(docs)` right after loading.
- **Case 6** is why you keep the embedding model in one file (`embed.py`). If the dimensions had matched, you would get **no error and wrong results**.
- **Case 9** shows why stable ids matter. Duplicates make the same chunk appear several times in the results and push out other useful chunks.

10.3 Troubleshooting table Beginner

Symptom	Likely cause	Fix
`RuntimeError: Error loading <file>`	Wrong path, or the file is not readable in that encoding	Check the path. Use `encoding="utf-8"`. Look at `error.__cause__` for the real reason
`FileNotFoundError: Directory not found: 'data'`	You ran the script from a different folder, or `make_data.py` has not run	Run it from the project folder. Run `make_data.py` first
`DirectoryLoader` returns `[]`	The glob matches no file, or the working folder is not what you think	Print `os.getcwd()`. Test the glob with `Path("data").glob("**/*.md")`
`PyPDFLoader` gives empty text	The PDF is scanned images	Use OCR first
Garbled characters	Wrong encoding	`encoding="utf-8"`, or `autodetect_encoding=True` on `TextLoader`
`ValueError: Got a larger chunk overlap ...`	`chunk_overlap` is not smaller than `chunk_size`	Use an overlap well below the size
Chunks are cut in odd places	Wrong splitter for the file type	Use the Markdown or code splitter (section 3.5)
Chroma: `Expected a name containing 3-512 characters`	Collection name is too short	Use 3 or more characters
Chroma: `Collection expecting embedding with dimension of X, got Y`	The embedding model changed	Use the same model, or delete `chroma_db` and re-index
Chroma: `attempt to write a readonly database`	You deleted `chroma_db` while another program still had it open	Stop every program that uses the store, then rebuild
Results repeat the same text	Duplicates in the store, or near-identical chunks	Use ids. Rebuild. Try MMR
`ValueError: ... pickle ...` when loading FAISS	The safety flag is missing	`allow_dangerous_deserialization=True`, only for your own files
`NotImplementedError` from a threshold retriever	The store has no relevance scores (`InMemoryVectorStore`)	Use Chroma, or filter on your own scores
Threshold retriever returns `[]` for good questions	Threshold is too high for your model	Print scores. Lower the threshold
The first run is slow and then fine	The embedding model downloads once	Wait. Later runs use the saved model
Answers ignore the context	Prompt too weak, or too many chunks	Stricter rules. Lower `k`
The model answers questions that are not in the documents	No refusal rule	Add the rule (section 7.1). Add a threshold guard (section 9.4)
Follow-up questions find nothing	The search used the raw follow-up	Rewrite it to a standalone question (section 7.5)
`DeprecationWarning ... langchain-community is being sunset`	The community package is no longer maintained	It still works. Pin versions. Prefer provider packages (section 1.6)

Part 11: Practice, interview questions, cheat sheet and recap

11.1 Hands-on exercises Beginner

Try these without looking at the notes first. The hint column points to the section to re-read.

#	Task	Hint	You are done when
1	Create a `Document` about shipping with two metadata fields and print it	2.1	You see the text and both fields
2	Load every `.md` file in `data/` and print the file name and length of each	2.2	Two lines appear
3	Split `refund_policy.txt` with `chunk_size` 100, 300 and 600 and print how many chunks each gives	3.2	Smaller size gives more chunks
4	Use `MarkdownHeaderTextSplitter` on `account_help.md` and print each section name	3.5	Three section names
5	Embed three sentences and find the most similar pair with `cosine`	4.3	The pair you expected
6	Build a Chroma store in a folder called `my_db`, end the program, start it again and search without re-adding	5.4	The second run finds results immediately
7	Retrieve for "delivery" with `similarity` and with `mmr` and compare the results	6.2	You can explain the difference
8	Add a new file `data/warranty.txt` (30 days warranty on electronics), rebuild the index and ask about it	8.2	The answer quotes the new file
9	Make `policy_bot.py` answer "I could not find that in our documents." **without calling the model** when nothing is retrieved	6.3, 9.4	An off-topic question costs no model call
10	Add the hybrid retriever to the bot and compare hit rates with `check_retrieval.py` before and after	6.5, 8.5	You have two sets of numbers

Solution for exercise 8.

```
# snip_ex8.py (run once, then run build_index.py again)
from pathlib import Path

Path("data/warranty.txt").write_text(
    "Nova Store Warranty\n\n"
    "All electronics have a 30 day warranty. If the product stops working within 30 days,\n"
    "contact support with the order number. We repair or replace it for free.\n",
    encoding="utf-8",
)
```

The loader already finds it (`**/*.txt`). Optionally add `"warranty.txt": "policy"` to `KIND` in `kb.py`, so that its `kind` is `policy` instead of `other`. Run `python build_index.py`, then ask `policy_bot.py`: "How long is the warranty on a phone?"

Solution for exercise 9.

```python
# snip_ex9.py
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda

from llm import get_model
from rag_common import RAG_PROMPT, format_docs, source_label
from store import get_store

strict = get_store().as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.4, "k": 4},
)
answer_chain = (
    RunnableLambda(lambda x: {"context": format_docs(x["docs"]), "question": x["question"]})
    | RAG_PROMPT | get_model() | StrOutputParser()
)


def ask(question):
    docs = strict.invoke(question)
    if not docs:                                    # nothing good enough: no model call at all
        return "I could not find that in our documents.", []
    return answer_chain.invoke({"docs": docs, "question": question}), docs


print(ask("Who won the cricket match yesterday?")[0])
```

Choose the threshold by printing relevance scores for a few good and bad questions (section 5.5). The value `0.4` is only a starting guess.

11.2 Interview questions and answers Intermediate

RAG basics

Q1. What is RAG and why do we use it? Retrieval-Augmented Generation: for each question, retrieve the relevant text from your own data, add it to the prompt, and let the model answer from it. We use it because a model does not know private or recent data, cannot hold a whole knowledge base in one prompt, and should be able to show its sources.

Q2. What are the two phases of a RAG system? Indexing (load, split, embed, store). It runs once or when data changes. Querying (embed the question, retrieve, build the prompt, generate). It runs for every question.

Q3. RAG or fine-tuning? RAG adds **knowledge** and is easy to update and to cite. Fine-tuning changes **behaviour** (style, format, skills) and is poor at adding facts. Most knowledge assistants start with RAG.

Q4. Why not paste all documents into the prompt? Context limits, cost and latency on every call, noise that lowers answer quality, and no control over sources.

Q5. How do you reduce hallucinations in RAG? A strict "only the context" prompt, a clear refusal sentence, a retrieval score threshold, temperature 0, good retrieval, and showing sources.

Loaders and splitters

Q6. What is a `Document` in LangChain? An object with `page_content` (text) and `metadata` (a dictionary). It is the unit that loaders produce and that splitters, vector stores and retrievers pass around.

Q7. What does a document loader return? A list of `Document` objects. How many depends on the loader: `TextLoader` gives one per file, `PyPDFLoader` one per page, `CSVLoader` one per row.

Q8. Why do we split documents? Prompts have size limits, a vector for a whole file is vague, and small chunks make search precise and prompts cheap.

Q9. What are `chunk_size` and `chunk_overlap`? How do you choose them? The maximum chunk length and the text shared by neighbouring chunks. Start with 500 to 1000 characters and 10 to 20 percent overlap, then measure retrieval on test questions and adjust.

Q10. How does `RecursiveCharacterTextSplitter` work? It tries separators in order (paragraph, line, word, character) and splits at the largest one that keeps pieces under `chunk_size`, so paragraphs and sentences stay together as much as possible. Then it merges small pieces and adds the overlap.

Q11. Why is metadata important in RAG? For filtering (department, version, user access), for showing sources (file, page) and for debugging.

Embeddings

Q12. What is an embedding? A list of numbers (a vector) produced by a model so that texts with similar meaning have vectors that are close together.

Q13. What is the difference between `embed_query` and `embed_documents`? `embed_query` embeds one question. `embed_documents` embeds many chunks. Some models treat queries and passages differently, so the interface keeps them apart.

Q14. Why must the same embedding model be used for chunks and questions? Different models create different vector spaces. Mixing them gives an error when dimensions differ and silently wrong results when they do not. Changing the model means re-indexing.

Q15. What is cosine similarity? A measure of the angle between two vectors. 1 means the same direction (same meaning). It ignores vector length.

Q16. What are the weaknesses of embeddings? Exact strings such as codes and names, negation, rare jargon, numbers, and long text that blurs many ideas. Hybrid search and good chunking help.

Q17. How do you choose an embedding model? By language support, quality on your own questions, input length limit, vector size, cost and privacy. A local model is free and private. An API model is often stronger.

Vector stores and retrievers

Q18. What is a vector store? A database that stores chunk text, vectors and metadata, and finds the chunks closest to a query vector. Examples: Chroma, FAISS, pgvector, Qdrant, Pinecone.

Q19. What is the difference between a vector store and a retriever? A vector store is the database with many methods. A retriever is a Runnable that takes a string and returns documents. `store.as_retriever()` turns one into the other, and the retriever can be piped into chains.

Q20. How does Chroma keep data between runs? What if you do not give ids? With `persist_directory`, it saves to a folder. Without ids, adding the same chunks again creates duplicates. With stable ids, the same chunk is updated, not duplicated.

Q21. Why does `FAISS.load_local` need `allow_dangerous_deserialization=True`? The saved docstore is a Python pickle, which can execute code when loaded. You set the flag only for files you created yourself.

Q22. Compare similarity, MMR and score-threshold retrieval. Similarity returns the `k` closest chunks. MMR picks results that are relevant and different from each other. Score threshold returns only chunks above a relevance score, which can be none.

Q23. How do you choose `k`? Start with 3 to 5. A larger `k` raises the chance that the right chunk is included but adds noise and cost. Measure hit rate on test questions.

Q24. What is hybrid search and why use it? Combining keyword search (BM25) with vector search, usually with `EnsembleRetriever` and reciprocal rank fusion. Keywords catch exact terms, and vectors catch meaning.

Q25. How do you filter retrieval by metadata, and why? Pass a `filter` (a dictionary in Chroma, a function in `InMemoryVectorStore`). Use it for departments, versions and access control. Filter in the retrieval step. Never rely on the model to hide documents.

Q26. What is `MultiQueryRetriever`? It asks the model to rewrite the question in several ways, searches with each, and merges the unique results. It helps with vague questions and costs one extra model call.

Design and operations

Q27. How do you write a RAG chain with LCEL? What does the dictionary do? `{"context": retriever | format_docs, "question": RunnablePassthrough()} | prompt | model | StrOutputParser()`. The dictionary is a `RunnableParallel`: it builds the two prompt inputs from the one question.

Q28. How do you handle follow-up questions in RAG? Rewrite the follow-up into a standalone question using the chat history, retrieve with that, then answer.

Q29. An answer is wrong. How do you debug it? Print the retrieved chunks. If the right chunk is missing, fix retrieval (chunks, `k`, hybrid, embeddings). If it is present but ignored, fix the prompt or the model. If the fact is not in the documents, fix the documents.

Q30. How do you evaluate retrieval? Build a list of questions with the document that should answer each one. Measure hit rate at k (and rank). Compare numbers before and after each change.

Q31. How do you keep the index up to date? Use stable ids so unchanged chunks are not duplicated, delete the chunks of changed files and add the new ones, or rebuild when you change the splitter or the embedding model.

Q32. What is prompt injection in RAG, and how do you reduce the risk? A retrieved document contains instructions that the model may follow. Treat context as data, review what you index, limit what the model can do, and filter documents by user permission at retrieval time.

11.3 One-page cheat sheet Beginner

Task	Code
Make a Document	`Document(page_content="...", metadata={"source": "..."})`
Load a text file	`TextLoader("f.txt", encoding="utf-8").load()`
Load a folder	`DirectoryLoader("data", glob="**/*.md", loader_cls=TextLoader, loader_kwargs={"encoding": "utf-8"}).load()`
Load a PDF (one Document per page)	`PyPDFLoader("f.pdf").load()`
Load a CSV (one Document per row)	`CSVLoader("f.csv", encoding="utf-8").load()`
Split Documents	`RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=80).split_documents(docs)`
Split Markdown by headings	`MarkdownHeaderTextSplitter([("#", "title"), ("##", "section")]).split_text(md)`
Embedding model	`FastEmbedEmbeddings(model_name="BAAI/bge-small-en-v1.5")`
Embed	`emb.embed_query(q)`, `emb.embed_documents([...])`
In-memory store	`InMemoryVectorStore.from_documents(chunks, emb)`
Chroma on disk	`Chroma(collection_name="kb", embedding_function=emb, persist_directory="chroma_db")`
Add with ids	`store.add_documents(chunks, ids=[...])`
Search	`store.similarity_search(q, k=4)`
Search with scores	`store.similarity_search_with_score(q, k=4)`
Metadata filter (Chroma)	`store.similarity_search(q, filter={"kind": "faq"})`
Retriever	`store.as_retriever(search_kwargs={"k": 4})`
MMR	`store.as_retriever(search_type="mmr", search_kwargs={"k": 4, "fetch_k": 12})`
Threshold	`store.as_retriever(search_type="similarity_score_threshold", search_kwargs={"score_threshold": 0.5})`
Keyword search	`BM25Retriever.from_documents(chunks, k=4)`
Hybrid	`EnsembleRetriever(retrievers=[bm25, vector], weights=[0.4, 0.6])`

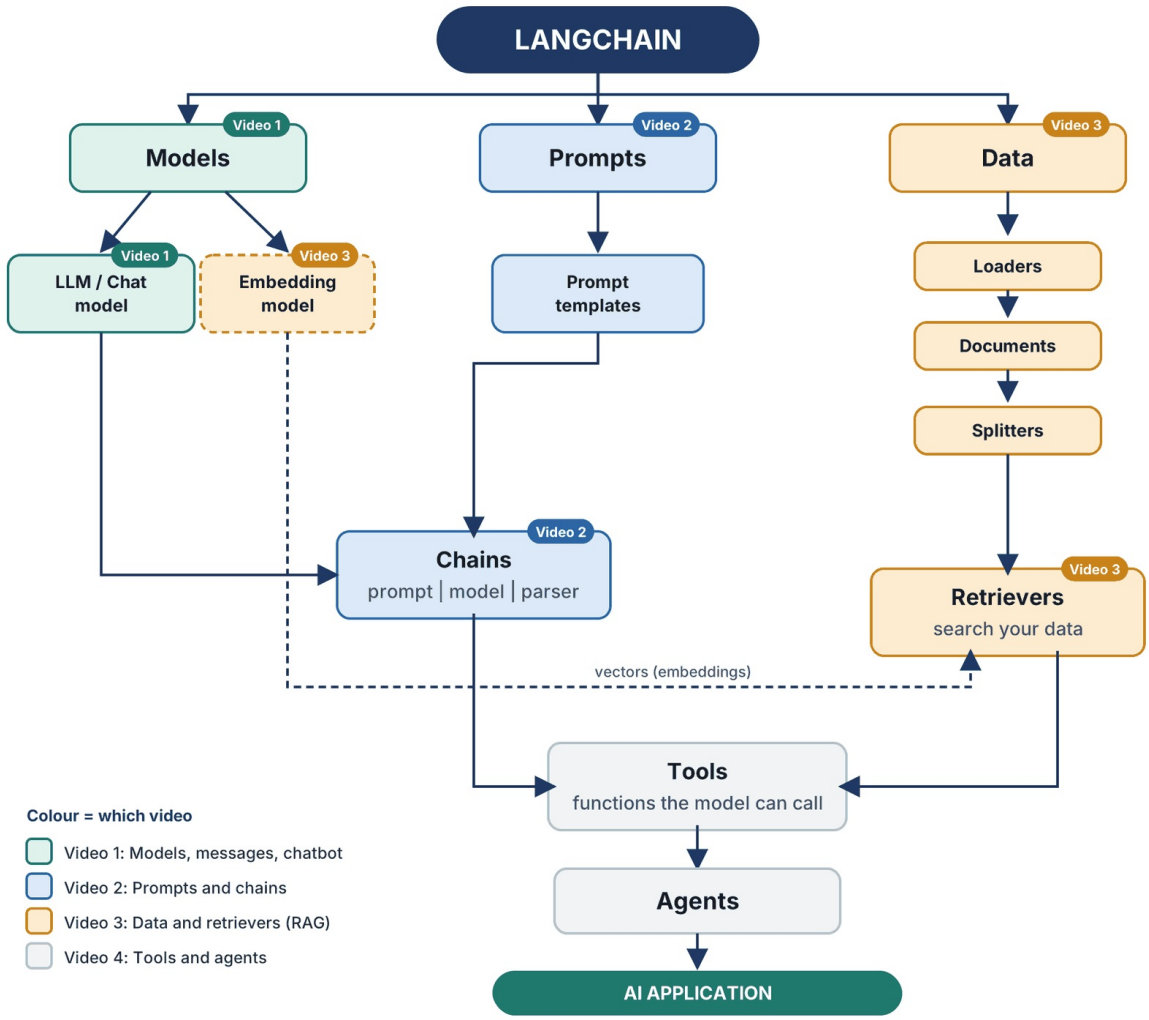
RAG chain	`{"context": retriever | format_docs, "question": RunnablePassthrough()} | prompt | model | StrOutputParser()`
Keep the sources	`{"docs": retriever, "question": RunnablePassthrough()} | RunnablePassthrough.assign(answer=answer_chain)`

The shape to remember:

```
Indexing:  files -> Documents -> chunks -> vectors -> vector store
Querying:  question -> retriever -> chunks -> prompt -> model -> answer + sources
```

11.4 Final recap Intermediate

Green and blue are Videos 1 and 2. The Video 3 blocks are now lit too. Only the last video is grey.



Your position on the map after Video 3.

By the end of this video, you have:

- Understood **RAG** and its two phases: indexing and querying.
- Loaded text, Markdown, PDF and CSV files into **Documents**, with useful metadata.
- Cut documents into **chunks** with `chunk_size`, `chunk_overlap` and structure-aware splitters.
- Explained **embeddings** and compared text by meaning.
- Stored and searched chunks in **InMemoryVectorStore**, **Chroma** and **FAISS**, with ids and filters.
- Used **retrievers**: top-k, MMR, score threshold, BM25 and hybrid search.
- Built a **RAG chain** that answers from your documents, refuses when it should, and shows sources.
- Added **chat history** to RAG with question rewriting.
- Built the **Nova Store policy assistant**, upgraded the ticket assistant, and measured retrieval with a hit

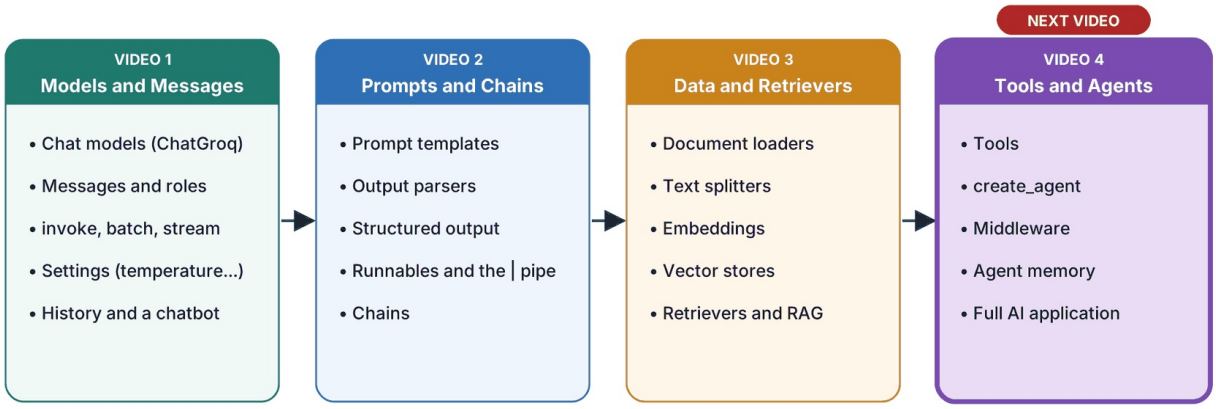
rate.

- Learned a debugging routine and the main failure causes.

Five sentences to remember:

1. RAG means: find the right text first, then let the model answer from it.
2. Good answers start with good **loading, chunking and metadata**. Check them before anything else.
3. Use the **same embedding model** for chunks and questions. Change the model, rebuild the index.
4. When an answer is wrong, **print the retrieved chunks** first.
5. Measure retrieval with test questions, and change one thing at a time.

11.5 Next video Intermediate



Next: Video 4, Tools and Agents.

Video 4: Tools and Agents. So far our app follows a fixed path: retrieve, then answer. In Video 4 the **model decides what to do**. You give it tools: search the policy, look up an order, calculate a refund date. It chooses which tool to call and when. You will build tools, use `create_agent`, add middleware and memory, and finish the **full AI application** at the bottom of the map. Your retriever from this video becomes one of the agent's tools.

11.6 Useful links Intermediate

- LangChain documentation: https://docs.langchain.com
- Chroma documentation: https://docs.trychroma.com
- FAISS (Meta): https://github.com/facebookresearch/faiss
- FastEmbed: https://github.com/qdrant/fastembed
- Embedding model leaderboard (MTEB): https://huggingface.co/spaces/mteb/leaderboard
- Groq model list: https://console.groq.com/docs/models